



CU Online

Discover. Learn. Empower.



Spike-Sense: AI-Driven Anomaly Detection for Cloud Infrastructure Metrics

Submitted in partial fulfillment of the requirements for the award of the degree
of

Master of Computer Applications (MCA)

By

Student Name: Mayank Mohan

Enrollment No: O24MCA*** Registration No: CUOL885086

Under the guidance of

Guide/Mentor Name: Vasanthi Chandran

Certificate

Copy to the certificate received from Qollabb to be pasted here.

Declaration

I, Mayank Mohan, hereby solemnly declare that the project report titled “Spike-Sense: AI-Driven Anomaly Detection for Cloud Infrastructure Metrics” submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2024–26 under the supervision of Vasanthi Chandran (Guide/Mentor Name).
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: _____

Date: _____

Student Signature: _____ Student Name: Mayank Mohan Enrollment
No: O24MCA*** Registration No: CUOL885086

Acknowledgement

I wish to express my sincere gratitude to my guide and mentor, Vasanthi Chandran, whose guidance, technical insight and constant encouragement shaped this project from its first design sketch to the final working system. Her feedback on the evaluation methodology and on presenting results honestly was especially valuable. I am thankful to the faculty of the Centre for Distance and Online Education, Chandigarh University, for providing a rigorous curriculum that made a project of this scope possible, and for the timely academic support extended throughout the semester. I gratefully acknowledge the Numenta Anomaly Benchmark (NAB) project for making real, labelled cloud telemetry openly available, without which the empirical evaluation in this report could not have been performed. Finally, I thank my family and peers for their patience and support during the development and writing of this work.

Abstract / Executive Summary (150–250 words)

- Topic of the study
- Objective
- System Analysis
- System Design
- System Implementation
- Testing
- Future Scope

Table of Contents

Certificate.....	1
Declaration.....	2
Acknowledgement.....	3
Abstract / Executive Summary (150–250 words).....	4
List of Figures.....	8
List of Tables.....	10
List of Abbreviations.....	11
Abstract.....	12
Chapter 1: Introduction.....	13
1.1 Background of the Project.....	13
1.2 Problem Statement.....	15
1.3 Objectives of the System.....	16
1.4 Scope of the Project.....	17
1.5 Existing System and Its Limitations.....	18
1.6 Proposed System Overview.....	18
1.7 Technologies Used (Brief Introduction).....	19
1.8 Significance of the Study.....	19
1.9 Organisation of the Report.....	20
Chapter 2: System Study.....	21
2.1 Introduction to the Domain.....	21
2.2 Categories of Anomalies in Time-Series.....	21
2.3 Review of Detection Approaches.....	21
2.4 The Numenta Anomaly Benchmark Dataset.....	23
2.5 Software Development Methodology.....	25
2.6 Relevant Frameworks and Libraries.....	25
2.7 Research Gap and Motivation.....	27
Chapter 3: System Analysis.....	28
3.1 Introduction.....	28
3.2 Functional Requirements.....	28
3.3 Non-Functional Requirements.....	29
3.4 User Requirements.....	30
3.5 Use-Case Analysis.....	30
3.6 System Architecture (Analysis View).....	31
3.7 Feasibility Study.....	32

3.8 Data-Flow Diagrams.....	33
3.9 Summary.....	35
Chapter 4: System Design.....	36
4.1 Introduction.....	36
4.2 Architectural Design.....	36
4.3 Class Design.....	37
4.4 Database Design.....	39
4.5 API Design.....	42
4.6 Dynamic Behaviour.....	43
4.7 Design of the Detection Models.....	46
4.8 Summary.....	46
Chapter 5: System Implementation.....	47
5.1 Introduction.....	47
5.2 Development Environment.....	47
5.3 Tools and Technologies Used.....	47
5.4 Hardware and Software Requirements.....	48
5.5 Module-Wise Implementation.....	48
5.6 Security Considerations.....	59
5.7 Version Control.....	60
5.8 Summary.....	60
Chapter 6: Testing.....	61
6.1 Introduction.....	61
6.2 Testing Strategy.....	61
6.3 Unit Testing.....	61
6.4 Integration Testing.....	62
6.5 Representative Test Cases.....	63
6.6 Test Results and Coverage.....	63
6.7 Summary.....	64
Chapter 7: Results & Discussion.....	65
7.1 Introduction.....	65
7.2 Training Behaviour.....	65
7.3 Test-Set Performance.....	66
7.4 Confusion Matrices.....	67
7.5 Precision–Recall Analysis.....	68
7.6 Hyper-Parameter Sensitivity.....	69

7.7 Controlled Synthetic-Anomaly Study.....	71
7.8 Comparison with the Existing System.....	72
7.9 Improvements Achieved and Live Demonstration.....	72
7.10 Discussion.....	74
7.11 Summary.....	75
Chapter 8: Conclusion & Future Scope.....	76
8.1 Conclusion.....	76
8.2 Achievement of Objectives.....	76
8.3 Key Achievements.....	76
8.4 Technical Learning Outcomes.....	77
8.5 Limitations.....	77
8.6 Future Scope.....	78
8.7 Concluding Remark.....	78
Chapter 9: References.....	79
Chapter 10: Appendices.....	81
Appendix A — Configuration.....	81
Appendix B — Project Structure.....	81
Appendix C — Installation Guide.....	82
Appendix D — User Manual.....	82
Appendix E — Source Code Availability.....	82

List of Figures

Figure 1.1 High-level architecture of Spike-Sense, showing the data, model, service and presentation layers and the flow of telemetry from raw CSV series to dashboard, database and alerts.....	15
Figure 2.1 A real NAB series, <code>cpu_utilization_asg_misconfiguration</code> , with its officially labelled anomaly window shown in red. The labelled fault falls in the final portion of the timeline, which under a chronological split becomes the held-out test data.....	24
Figure 3.1 Use-case diagram showing the operations analyst and evaluator actors, the external Discord service, and the use cases provided within the Spike-Sense system boundary	31
Figure 3.2 Level-0 (context) data-flow diagram. The system is a single process exchanging data with the NAB dataset, the analyst, and the Discord service.....	33
Figure 3.3 Level-1 data-flow diagram decomposing the system into its six processes and four data stores.....	34
Figure 4.1 Class diagram of the principal classes and structured records, including the model registry, the preprocessing and split records, the alert-state tracker, and the two persisted entities.....	38
Figure 4.2 Entity–relationship diagram of the persistence layer. The PREDICTIONS entity is related one-to-many to the ALERTS entity through the <code>prediction_id</code> foreign key.....	40
Figure 4.3 Sequence diagram of single-window scoring, from the dashboard request through scaling, both models, combination, persistence and alerting, to the response.....	43
Figure 4.4 Sequence diagram of the spike-injection demonstration, showing injection, preprocessing, batch scoring and persistence of the triggering detection.....	44
Figure 4.5 Activity diagram of the training pipeline, from per-series preprocessing through aggregation to the training and persistence of both models.....	45
Figure 5.1 Automatically generated Swagger (OpenAPI) documentation of the Spike-Sense REST API, showing the System, History, Inference, Evaluation and Demo endpoint groups and the data schemas.....	54
Figure 5.2 The Spike-Sense dashboard displaying a real EC2 CPU series. The top panel shows the metric with ground-truth anomalies; the lower panels show the two detectors' signals.....	56
Figure 5.3 The dashboard after injecting a point spike. The detectors flag the injected region; the lower panels show the corresponding rise in Isolation Forest score and LSTM reconstruction error.....	57
Figure 5.4 JSON response of the <code>/alerts</code> endpoint, listing recent persisted alerts with their detecting models and delivery status.....	58

Figure 7.1 LSTM Autoencoder training and validation loss per epoch. Training ran for six epochs under early stopping; the lowest validation loss occurs at the first epoch.....	65
Figure 7.2 Test-set precision, recall and F1-score for the four configurations. The union ensemble attains the highest F1 and recall.....	67
Figure 7.3 Confusion matrices for the four configurations on the test set, showing the true/false positive and negative counts behind the headline metrics.....	68
Figure 7.4 Precision–recall curves for the four configurations with their average-precision (AP) scores. The combined configurations achieve the highest AP.....	69
Figure 7.5 Isolation Forest performance on validation data as the contamination parameter varies; lower values trade recall for substantially higher precision.....	70
Figure 7.6 LSTM Autoencoder performance on validation data as the threshold percentile varies, showing the precision–recall trade-off the threshold controls.....	70
Figure 7.7 Recall of each configuration on injected synthetic anomalies. The Isolation Forest detects point and level-shift anomalies with perfect recall and trend drift strongly.....	71
Figure 7.8 The dashboard’s evaluation panel, showing the live test-set performance table together with the metric and detector charts.....	73

List of Tables

Table 2.1 Comparison of the two detectors adopted in Spike-Sense.....	23
Table 3.1 Functional requirements of the Spike-Sense system.....	28
Table 4.1 Database schema: columns of the predictions and alerts tables.....	41
Table 4.2 REST API endpoints exposed by the FastAPI service layer.....	42
Table 5.1 Tools and technologies used in the implementation, with versions and roles.....	47
Table 6.1 Representative test cases and their outcomes (selected from 114 automated tests)	63
Table 7.1 Test-set performance of the four detector configurations on 5,564 windows.....	66
Table 10.1 Principal configuration parameters.....	81

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
AIOps	Artificial Intelligence for IT Operations
API	Application Programming Interface
AWS	Amazon Web Services
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data-Flow Diagram
EC2	Elastic Compute Cloud (AWS)
ER	Entity–Relationship
F1	F1-score (harmonic mean of precision and recall)
FPR	False-Positive Rate
HTTP	HyperText Transfer Protocol
IF	Isolation Forest
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
ML	Machine Learning
MSE	Mean Squared Error
NAB	Numenta Anomaly Benchmark
ORM	Object–Relational Mapping
PR	Precision–Recall
RDS	Relational Database Service (AWS)
REST	Representational State Transfer
RMS	Root Mean Square
SQL	Structured Query Language
TOC	Table of Contents
UML	Unified Modeling Language
YAML	YAML Ain't Markup Language

Abstract

Spike-Sense is an AI-driven anomaly detection system for cloud infrastructure metrics. Modern cloud services emit continuous streams of telemetry — CPU utilisation, network throughput, request latency — in which early-warning signals of failure are easily lost amid normal fluctuation. The objective of this project is to detect such anomalies automatically and surface them to an operator in real time, without manually configured thresholds. The system combines two complementary unsupervised detectors: an Isolation Forest, which isolates outliers from eight statistical features of a sliding window, and an LSTM Autoencoder, which flags windows it cannot reconstruct well after being trained only on normal behaviour. Their outputs are fused to trade recall against precision.

The system was developed in Python with an incremental, test-driven methodology. A FastAPI service serves the pre-trained models through REST endpoints for scoring, evaluation and a live spike-injection demonstration; a Streamlit dashboard visualises the metric stream with both detectors' scores; a SQLite database persists every detection and alert; and a Discord webhook delivers notifications. The models were trained and evaluated on six real Amazon CloudWatch series from the Numenta Anomaly Benchmark, labelled with NAB's official anomaly windows and split chronologically to prevent leakage. On a held-out test set of 5,564 windows (1,784 anomalous), the union ensemble was best balanced — precision 0.49, recall 0.53, F1-score 0.51 — and 114 automated tests pass at 88% coverage. The result is a complete, reproducible, zero-cost pipeline showing that an ensemble of classical and deep-learning detectors gives practical, explainable monitoring of cloud metrics.

Chapter 1: Introduction

1.1 Background of the Project

Cloud computing has become the default operating environment for modern software. Compute, storage, networking and managed databases are rented on demand and scaled elastically, and the health of these resources is observed through telemetry: numeric measurements sampled at regular intervals and recorded as time-series. A single virtual machine may emit dozens of such series — processor utilisation, memory pressure, disk throughput, inbound and outbound network traffic — and a production deployment of even modest size produces thousands of them. Amazon CloudWatch, Azure Monitor and Google Cloud Operations are the dominant platforms that collect, store and visualise this data.

Within these streams, the events that matter most for reliability are anomalies: points or intervals where the metric departs from its established pattern. A processor that has hovered around forty percent for a fortnight and then climbs to a sustained ninety-five percent, a latency metric that suddenly doubles, or a network interface that goes silent are all signs of an underlying problem — a misconfigured auto-scaling group, a memory leak, a failing disk, or an external attack. Detecting these deviations quickly is the foundation of observability and of the discipline now known as AIOps, the application of machine learning to IT operations.

Spike-Sense is a project built around this problem. It ingests real cloud telemetry, learns what normal behaviour looks like for each metric, and reports when the incoming values no longer fit that learned profile. The name reflects the most visually obvious form of anomaly — a spike — although the system is designed to catch sustained level shifts and gradual drifts as well as sharp transient spikes.

The economic stakes behind this seemingly narrow technical problem are considerable. Industry surveys routinely place the cost of unplanned infrastructure downtime in the thousands of currency units per minute for medium-sized services, and far higher for large platforms, because an outage simultaneously halts revenue, breaches service-level agreements and damages user trust. The decisive variable in containing that cost is the mean time to detection: the interval between the moment a fault begins to manifest in the metrics and the moment a human or an automated system notices it. Every minute shaved from detection is a minute earlier that remediation can begin. Anomaly detection, then, is not an academic curiosity but

the first link in the incident-response chain, and improving it has a direct and measurable operational value. This is the practical backdrop against which Spike-Sense is designed.

The discipline that has grown around this need is called observability, and its more automated, machine-learning-assisted form is increasingly referred to as AIOps. Where traditional monitoring answered the question ‘is this specific metric above its limit?’, observability asks the broader question ‘is the system behaving the way it normally does?’ — a question that static limits cannot answer but that a model of normal behaviour can. Spike-Sense sits squarely within this shift, applying learned models of normality to the concrete and well-bounded case of univariate cloud metric streams.

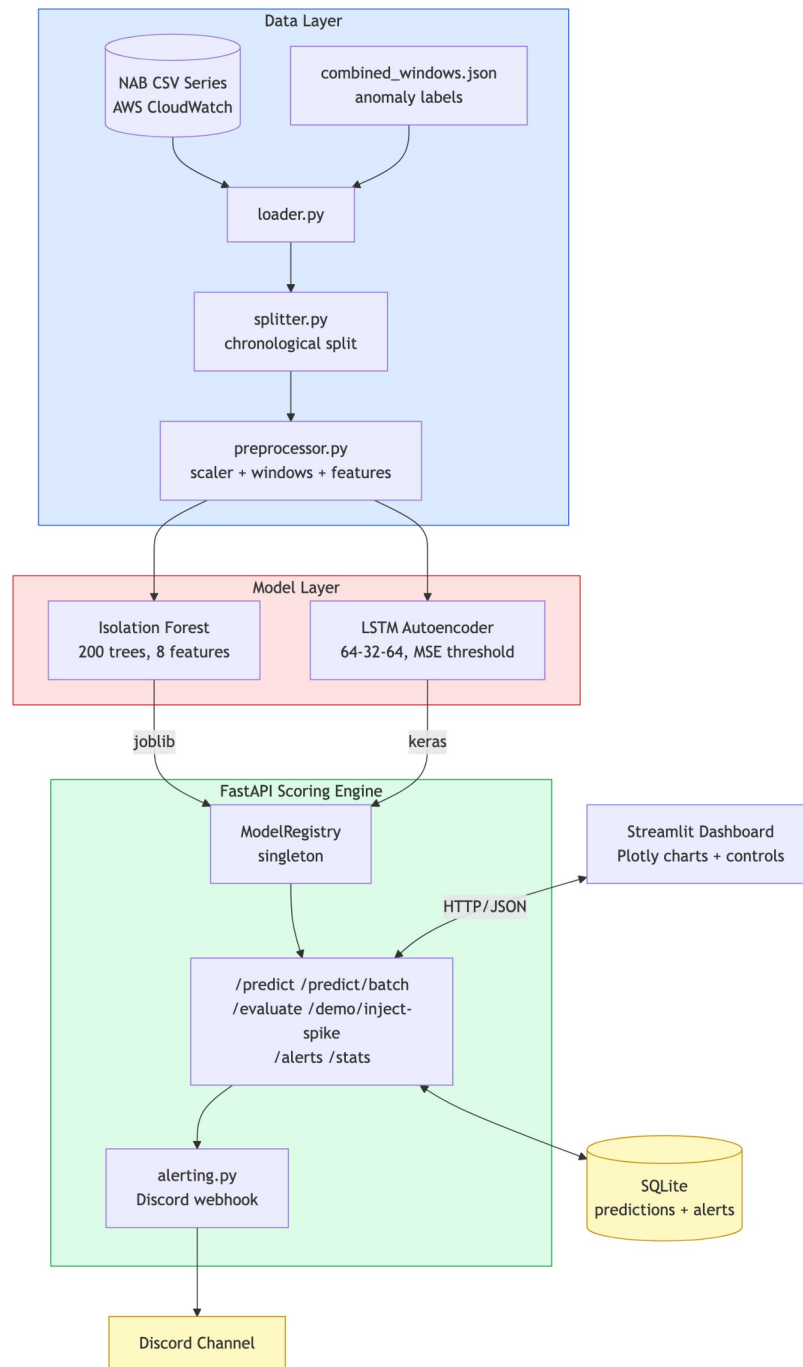


Figure 1.1: High-level architecture of Spike-Sense, showing the data, model, service and presentation layers and the flow of telemetry from raw CSV series to dashboard, database and alerts.

1.2 Problem Statement

Operations teams face three persistent difficulties when monitoring metric streams. First, the volume of telemetry far exceeds what a human can watch directly; a person cannot keep thousands of charts under continuous observation. Second, the conventional automated alternative — static thresholds, where an alert fires when a metric crosses a fixed value — is

brittle: a threshold appropriate for daytime traffic produces false alarms or silent misses at night, and a threshold for one server is wrong for another. Third, the anomalies that matter are heavily outnumbered by normal observations, so any detector must operate under severe class imbalance where naively predicting ‘normal’ achieves high accuracy yet is operationally useless.

The problem this project addresses can therefore be stated precisely: given a univariate stream of cloud metric values sampled at fixed intervals, automatically and without manual threshold tuning, identify the windows of time that represent anomalous behaviour, present the verdict to an operator with enough context to act on it, and do so in a way that can be evaluated objectively against known ground truth.

A further dimension of the problem is the imbalance between the two kinds of error a detector can make. A false negative — a missed anomaly — can allow an incident to escalate unchecked, while a false positive — a false alarm — erodes trust and, if frequent enough, trains operators to ignore the system altogether. The relative cost of these two errors differs by situation, so a useful detector must not commit to a single fixed operating point but should expose a way to trade one error against the other. Spike-Sense addresses this directly: its ensemble offers union and intersection combinations at opposite ends of the trade-off, and each model’s threshold can be tuned along a continuum, so the operating point can be chosen to match the cost structure of a given deployment rather than imposed by the tool.

1.3 Objectives of the System

The project set out to meet the following concrete objectives:

1. To build a reproducible data pipeline that loads real cloud telemetry, attaches ground-truth anomaly labels, and transforms raw values into the windowed representations required by the detection models.
2. To implement two complementary unsupervised anomaly detectors — an Isolation Forest operating on engineered statistical features and an LSTM Autoencoder operating on raw windows — and to combine their outputs.
3. To expose the trained models through a documented REST API capable of scoring individual windows, scoring full series in batch, and running a controlled demonstration.
4. To provide an interactive dashboard that visualises the metric together with each model’s anomaly signal and lets a user inject synthetic anomalies to observe the system’s response.

5. To persist every detection and alert in a relational database so that detection history can be queried and audited.
6. To deliver real-time alerting through a Discord webhook with flooding protection.
7. To evaluate the system rigorously on real labelled anomalies using precision, recall, F1-score and precision–recall curves, and to validate the implementation with an automated test suite.

1.4 Scope of the Project

The scope of Spike-Sense is deliberately focused so that every component could be built, tested and evaluated to a working standard within a single semester at zero infrastructure cost. The system operates on univariate time-series — one metric at a time — which covers the large majority of practical CloudWatch monitoring use cases. It targets offline and near-real-time scoring of pre-recorded or replayed series rather than ingestion directly from a live cloud account, although the REST interface is designed so that a streaming producer could be added without changing the models.

Included in scope are: data loading and labelling from the Numenta Anomaly Benchmark; preprocessing, windowing and feature extraction; training, persistence and serving of both models; ensemble combination; the REST API; the Streamlit dashboard; the SQLite persistence layer; Discord alerting; controlled synthetic-anomaly injection for demonstration; and a full evaluation harness. Out of scope, and discussed in the chapter on future work, are multivariate correlation across several metrics, online incremental retraining, authentication and multi-tenant access control, and direct integration with a live cloud provider's monitoring API.

Fixing the scope in this way was itself a design decision that made the project completable to a working standard. By committing to univariate, replayed series and a defined set of metrics, the project avoided the open-ended complexity of multivariate correlation and live cloud integration while still exercising every stage of a genuine detection pipeline — loading, labelling, preprocessing, training, serving, persisting, alerting and visualising. The interfaces were nonetheless kept general enough that the excluded concerns could be added later without rework: the scoring endpoint would accept windows from a live streaming producer just as readily as from the dashboard. Scope was therefore used to bound effort without bounding extensibility, which is the balance a time-limited but ambitious project must strike.

1.5 Existing System and Its Limitations

The prevailing approach to metric monitoring in industry combines visualisation tools such as Grafana with time-series databases such as Prometheus or the native CloudWatch console, and layers static or simple rule-based alerts on top. These tools are mature and excellent at storage and display, but the detection logic they ship with is overwhelmingly threshold based. The limitations of that model are well known: thresholds must be set and maintained by hand for every metric; they cannot adapt to daily or weekly seasonality; they detect only magnitude violations and are blind to changes in shape or temporal pattern; and they generate alert fatigue when set sensitively or miss real incidents when set conservatively.

More advanced commercial offerings do incorporate machine learning, but they are typically proprietary, costly, and opaque — an operator cannot inspect why a detection was made. There is therefore a clear gap for an open, explainable, low-cost system that learns normal behaviour automatically and exposes its reasoning, which is the gap Spike-Sense targets.

1.6 Proposed System Overview

Spike-Sense replaces hand-tuned thresholds with two learned models. The Isolation Forest is a fast, tree-based outlier detector that requires no labels and yields an interpretable anomaly score; it works on eight statistical descriptors computed over each sliding window. The LSTM Autoencoder is a recurrent neural network trained to reconstruct only normal windows, so that windows it reconstructs poorly — measured by reconstruction error — are flagged as anomalous. Because the two models make errors in different ways, their combination is more robust than either alone: a union of their verdicts maximises recall, while an intersection maximises precision.

The rationale for using two models rather than one is central to the design. A single detector embodies a single set of assumptions and therefore a single characteristic blind spot. The Isolation Forest, operating on summary statistics of a window, excels at spotting windows whose overall magnitude or spread is unusual, but by reducing a window to eight numbers it discards the order of the samples and so is comparatively insensitive to anomalies that lie in the temporal shape of the data. The LSTM Autoencoder, operating on the raw ordered sequence, is sensitive precisely to that shape but is more conservative about isolated magnitude spikes. By running both and combining their verdicts, the system covers the weaknesses of each: this complementarity, and its empirical confirmation in Chapter 7, is the central hypothesis of the project.

Around this detection core the project provides a complete, deployable application: a FastAPI scoring service, a Streamlit operator dashboard, a SQLite detection-history database, and Discord alerting, all of which are described in detail in the chapters that follow. The emphasis on delivering a whole working system, rather than an isolated model in a notebook, is deliberate: it is the difference between demonstrating that a technique can detect anomalies and demonstrating that it can be operated.

1.7 Technologies Used (Brief Introduction)

The system is implemented entirely in Python. Scikit-learn provides the Isolation Forest; TensorFlow and Keras provide the LSTM Autoencoder; NumPy, pandas and SciPy support data manipulation and feature engineering. FastAPI with Uvicorn serves the REST API, Pydantic validates its request and response schemas, and SQLAlchemy with SQLite persists detection history. Streamlit and Plotly build the interactive dashboard. Pytest with coverage drives the automated test suite. Each technology and the rationale for selecting it is examined in the implementation chapter.

Each technology was chosen for a specific reason that the implementation chapter develops. Python was selected because it is the lingua franca of data science and offers mature libraries for every layer of the system, allowing a single language to span data handling, machine learning, web services and visualisation. Scikit-learn and TensorFlow are the de-facto standards for classical and deep learning respectively, and are well tested and documented. FastAPI was preferred over older web frameworks because it derives validation and documentation directly from type annotations. SQLite was chosen over a client-server database because it needs no separate server and stores its data in a single file, matching the zero-cost and portability goals. Streamlit lets a rich interactive dashboard be written in pure Python. Together these choices keep the system lightweight, free and reproducible while remaining representative of current industry practice.

1.8 Significance of the Study

The significance of this work lies less in inventing a new algorithm than in demonstrating, end to end and at zero cost, that established techniques can be assembled into a coherent, explainable and operable monitoring system. For a practitioner, the project shows a concrete path from a public benchmark dataset to a running service with a dashboard, a database and alerts, and it documents the methodological care — leakage-free splitting, window-based

labelling, honest evaluation — that separates a credible result from a misleading one. For a student of computer applications, it integrates a broad cross-section of the discipline: data engineering, classical machine learning, deep learning, REST API design, relational data modelling, interactive visualisation and software testing, all within a single coherent artefact. The deliberate constraint of zero cost is itself significant, because it makes the entire system reproducible by anyone with a laptop, with no dependence on paid services or proprietary data.

1.9 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 surveys the relevant literature, datasets and methodologies. Chapter 3 presents the system analysis, including functional and non-functional requirements, a feasibility study, data-flow diagrams and use cases. Chapter 4 details the system design through UML diagrams, the database schema and the API design. Chapter 5 describes the implementation module by module with key code listings and screenshots. Chapter 6 sets out the testing strategy and test cases. Chapter 7 reports and discusses the empirical results. Chapter 8 concludes and outlines future enhancements. Chapter 9 lists the references, and Chapter 10 contains the appendices.

Chapter 2: System Study

2.1 Introduction to the Domain

Anomaly detection in time-series is a mature research area with a large body of literature spanning statistics, signal processing and machine learning. This chapter reviews the approaches most relevant to cloud-metric monitoring, examines the dataset on which Spike-Sense is trained and evaluated, surveys the software methodologies and libraries that inform the implementation, and identifies the gap that the project fills. The aim is not to reproduce textbook descriptions of each technique but to explain how each relates to the design decisions made in this system.

2.2 Categories of Anomalies in Time-Series

It is useful to distinguish the kinds of anomaly a detector might face, because different methods are suited to different kinds. A point anomaly is a single observation that is far from the rest, such as an isolated spike in CPU usage. A contextual anomaly is a value that is normal in general but abnormal in its context, for example ordinary daytime traffic seen at three in the morning. A collective anomaly is a sub-sequence of observations that is anomalous as a group even though no single point is extreme, such as a sustained level shift or a slow upward drift. Cloud telemetry exhibits all three, and Spike-Sense is evaluated against synthetic versions of each — point spike, level shift and trend drift — in addition to real labelled anomalies.

These categories are not merely taxonomic; they dictate which detection method is appropriate. A point anomaly is best caught by a method sensitive to the magnitude of individual observations relative to a learned distribution, which is the natural strength of a statistical or feature-based detector. A contextual or collective anomaly, by contrast, may contain no individually extreme value and is only abnormal as a pattern over time, so it demands a method that models temporal structure. No single technique is ideal for all three, which is the fundamental reason Spike-Sense pairs a feature-based detector with a sequence-based one rather than relying on either alone.

2.3 Review of Detection Approaches

2.3.1 Statistical and Threshold-Based Methods

The earliest and still most widespread methods are statistical. Fixed thresholds, moving averages, exponentially weighted moving averages and control charts flag values that fall

outside a band derived from recent history. These methods are transparent and cheap, but they assume a stationary distribution and a known scale, and they detect only magnitude violations. They form the baseline that learned detectors aim to surpass, and their weaknesses motivated the move to data-driven models in this project.

2.3.2 Tree-Based and Proximity Methods

Isolation Forest, introduced by Liu, Ting and Zhou, takes a different and elegant view of outliers: rather than profiling normal points, it isolates anomalies. It builds an ensemble of random binary trees that recursively partition the feature space along randomly chosen features and split values. Anomalous points, being few and different, are separated from the rest in fewer splits and therefore have shorter average path lengths from the root. The anomaly score is derived from this path length. The method is unsupervised, runs in near linear time, requires little tuning, and handles multi-dimensional feature vectors naturally — all properties that make it well suited to fast, explainable scoring of the eight statistical features Spike-Sense extracts per window. Related proximity methods such as the Local Outlier Factor and one-class support vector machines were considered but rejected because they scale less gracefully and offer less interpretable scores.

2.3.3 Deep Learning and Reconstruction Methods

Recurrent neural networks, and in particular the Long Short-Term Memory architecture of Hochreiter and Schmidhuber, can model temporal dependence in sequences through gated memory cells that mitigate the vanishing-gradient problem of plain recurrent networks. The LSTM cell maintains an internal state that is updated at each time step under the control of input, forget and output gates; these gates, which are themselves learned, decide what new information to store, what to discard and what to expose, allowing the network to carry relevant context across many steps without the gradient decaying to zero. This makes the LSTM well suited to telemetry, where the significance of a value often depends on the trajectory that preceded it.

An autoencoder is a network trained to reconstruct its own input through a narrow bottleneck that forces it to learn a compressed representation rather than memorise the data. Combining the two yields the LSTM Autoencoder used here. An encoder LSTM reads a window and compresses it into a fixed-length vector; this vector is repeated and passed to a decoder LSTM that attempts to regenerate the original window step by step. Trained only on normal windows, the network becomes proficient at reproducing ordinary temporal patterns, so that anomalous

windows — whose dynamics it has never learned to represent — are reconstructed poorly. The mean-squared reconstruction error therefore serves as a continuous anomaly score, and a threshold on that error converts it into a verdict. This reconstruction-based, semi-supervised framing is now a standard and effective approach to collective and contextual anomalies, and it complements the Isolation Forest’s strength at point anomalies.

It is worth noting why a purely supervised classifier was not used. Supervised learning would require a large, balanced and continuously maintained corpus of labelled anomalies, which is precisely what production telemetry lacks: anomalies are rare, diverse and often unprecedented, so a classifier trained on past anomalies tends to miss novel ones. The unsupervised and semi-supervised detectors chosen here instead model normality, which is abundant, and flag departures from it — an approach that generalises to anomaly types never seen during training. Table 2.1 summarises how the two selected detectors compare.

Table 2.1: *Comparison of the two detectors adopted in Spike-Sense.*

Property	Isolation Forest	LSTM Autoencoder
Learning paradigm	Unsupervised (isolation)	Semi-supervised (normal-only)
Input	8 statistical features per window	Raw ordered window
Captures	Magnitude / spread / shape stats	Temporal dynamics
Strongest on	Point and level anomalies	Contextual / collective anomalies
Training cost	Very low (seconds, CPU)	Moderate (minutes, CPU)
Interpretability	High (path-length score)	Moderate (reconstruction error)
Anomaly score	Negated decision function	Mean-squared reconstruction error

2.4 The Numenta Anomaly Benchmark Dataset

The choice of NAB over alternatives was deliberate. Synthetic datasets, in which anomalies are inserted artificially, make evaluation easy but prove little about real-world performance, because the inserted anomalies rarely resemble genuine faults. Purely unlabelled production data, on the other hand, offers realism but no ground truth against which to compute precision and recall. NAB occupies the valuable middle ground: it is real telemetry from operational systems, yet it is carefully hand-labelled by domain experts, and it is openly available without registration. This combination of realism and trustworthy labels is precisely what an honest evaluation of an anomaly detector requires, which is why NAB has become a standard reference benchmark in the streaming-anomaly-detection literature.

Reliable evaluation requires real data with trustworthy labels. Spike-Sense uses the Numenta Anomaly Benchmark (NAB), an openly available corpus created specifically to evaluate streaming anomaly detectors. NAB provides dozens of real and artificial time-series, each accompanied by hand-labelled anomalies. This project uses six real Amazon CloudWatch series from the benchmark’s realAWSCloudwatch and realKnownCause categories, covering processor utilisation, inbound network traffic and request latency, each sampled at five-minute intervals. Crucially, NAB labels anomalies not as single instants but as time windows around each known event, reflecting the operational reality that a fault occupies an interval. Spike-Sense adopts these official anomaly windows, marking every sample inside a window as anomalous, which yields a realistic positive rate of roughly eight to ten percent per series.

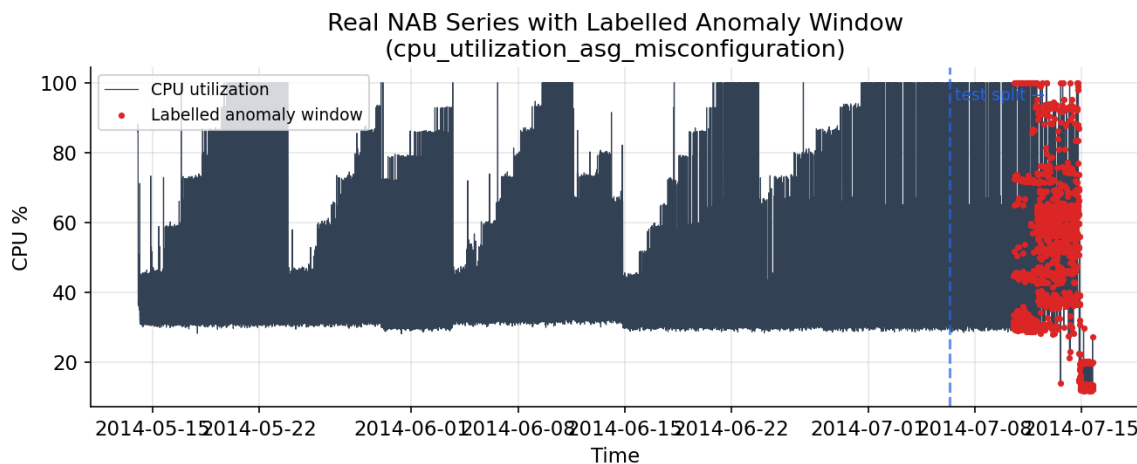


Figure 2.1: A real NAB series, *cpu_utilization_asg_misconfiguration*, with its officially labelled anomaly window shown in red. The labelled fault falls in the final portion of the timeline, which under a chronological split becomes the held-out test data.

The choice of series was deliberate. Because the system is evaluated under a chronological train/validation/test split designed to prevent information from the future leaking into training, it was important to include series whose labelled anomalies fall late in the timeline, so that genuine anomalies are present in the held-out test partition. Three of the six series satisfy this property, contributing 1,784 anomalous windows to the test set; the remaining series contribute additional normal behaviour and earlier anomalies that enrich training and validation.

Each series spans roughly two weeks of telemetry at five-minute resolution, giving on the order of four thousand samples per series, with the larger auto-scaling series running to around eighteen thousand. The six series were chosen to cover three distinct metric types — processor utilisation, inbound network traffic and request latency — and two failure categories within NAB: ordinary AWS CloudWatch streams and ‘known cause’ streams whose anomalies

correspond to documented incidents such as an auto-scaling misconfiguration or a system failure. This diversity matters because a detector that performs well only on one metric type or one failure mode would be of limited practical use; evaluating across several guards against that narrowness. Because each series has its own scale and baseline — processor percentages, byte counts and millisecond latencies are not comparable — a separate scaler is fitted per series, a design point taken up in the implementation.

2.5 Software Development Methodology

The project followed an incremental and iterative methodology with a strong test-first discipline, rather than a single linear waterfall pass. Each subsystem — the data pipeline, the two models, the evaluation harness, the API, the database and the dashboard — was specified, implemented and covered by automated tests before the next was begun, and earlier components were revisited as later ones revealed new requirements. This approach suited a research-flavoured project in which the evaluation methodology itself evolved as results were examined. Configuration was centralised in a single human-readable file so that hyper-parameters and dataset choices could be changed and the whole pipeline re-run reproducibly. Version control with Git tracked every change.

This methodology also shaped how risk was managed. The riskiest elements — whether the autoencoder would learn anything useful, and whether the evaluation would yield meaningful numbers — were tackled early, on small slices of data, before the surrounding application was built, so that a fundamental problem would surface while it was still cheap to address. Only once the detection core was demonstrably sound was effort invested in the service, persistence and presentation layers. This ordering, in which the most uncertain work is done first, is a deliberate hedge against the possibility that a late-discovered flaw in the core would invalidate work built on top of it.

2.6 Relevant Frameworks and Libraries

Several mature open-source libraries were adopted rather than re-implemented, both to follow industry practice and to keep the project's own code focused on its novel logic. Scikit-learn supplies a well-tested Isolation Forest; TensorFlow and Keras provide the building blocks and training loop for the autoencoder; FastAPI provides a high-performance, automatically documented web framework; SQLAlchemy provides a database-agnostic persistence layer; and Streamlit with Plotly enables a rich interactive dashboard with very little front-end code.

The role each plays is examined in the implementation chapter; here it suffices to note that the project's contribution lies in how these components are combined, configured and evaluated, not in the libraries themselves.

Using established libraries rather than re-implementing their functionality is itself a considered engineering choice. A hand-written Isolation Forest or LSTM would be more likely to contain subtle bugs, would lack the optimisation and testing of a widely used library, and would divert effort from the project's actual goal of building and evaluating a complete system. The literature is clear that reproducibility and correctness in machine-learning work are better served by relying on well-maintained, peer-reviewed implementations. The project therefore treats these libraries as trusted building blocks and concentrates its own code on the data pipeline, the ensemble logic, the service and persistence layers, the evaluation harness and the dashboard — the parts that are specific to Spike-Sense.

2.6.1 Evaluation Metrics for Imbalanced Detection

A review of methodology must also address how detection quality is measured, because the choice of metric profoundly affects what counts as success. Accuracy — the fraction of windows classified correctly — is actively misleading under the heavy class imbalance of anomaly detection, since a detector that predicts 'normal' for everything scores highly while being useless. The literature on imbalanced classification therefore favours precision, recall and their harmonic mean, the F1-score. Precision answers 'of the windows I flagged, how many were truly anomalous?', capturing the false-alarm burden. Recall answers 'of the truly anomalous windows, how many did I catch?', capturing missed incidents. The F1-score balances the two, and the false-positive rate measures the proportion of normal windows wrongly flagged.

Because any threshold-based detector trades precision against recall as its threshold moves, a single operating point tells an incomplete story. The precision–recall curve, which plots the achievable precision at every level of recall, and its summary the average precision, give a threshold-independent view of a detector's quality and are especially appropriate under imbalance. Spike-Sense reports all of these — precision, recall, F1, false-positive rate and precision–recall curves — so that its performance can be judged honestly rather than flattered by a single convenient number. This metric discipline, established here, is applied throughout the results chapter.

2.7 Research Gap and Motivation

The literature and the existing tools leave a clear gap. Threshold-based monitoring is transparent and cheap but inflexible and blind to pattern change. Sophisticated commercial AIOps platforms are adaptive but proprietary, costly and opaque. Academic detectors are powerful but are usually delivered as isolated models rather than as complete, operable systems. Spike-Sense occupies the gap between these: it is a fully working, openly constructed and explainable system that pairs a classical detector with a deep-learning one, serves them through a real API and dashboard, persists and alerts on detections, and is evaluated honestly on real labelled data — all at zero infrastructure cost. This combination of completeness, transparency and rigorous evaluation is the motivation for the project.

It is worth being precise about what is and is not novel here. The two detection algorithms are established, and the libraries that implement them are mature; the project does not claim to advance the state of the art in anomaly-detection algorithms. Its contribution is instead one of synthesis and engineering rigour: the careful combination of two complementary detectors into a tunable ensemble; the disciplined, leakage-free evaluation on real labelled anomalies using the benchmark's own scoring windows; and the delivery of the whole as an operable, tested, zero-cost system rather than a notebook experiment. In a field where a great many published results are difficult to reproduce and a great many student projects stop at a model that runs once, completeness and reproducibility are themselves worthwhile contributions, and they are the standard to which this project holds itself.

Chapter 3: System Analysis

3.1 Introduction

System analysis translates the broad objectives of Chapter 1 into precise, verifiable requirements and establishes that the proposed system is feasible. This chapter specifies the functional and non-functional requirements, identifies the users and their goals, examines technical, economic and operational feasibility, and models the flow of data through the system using data-flow diagrams and use-case analysis.

3.2 Functional Requirements

The functional requirements describe what the system must do. They were derived directly from the project objectives and refined as the implementation matured.

Table 3.1: *Functional requirements of the Spike-Sense system.*

ID	Requirement	Description
FR-1	Load and label data	Load NAB CSV series and attach binary anomaly labels from the official NAB anomaly-window definitions.
FR-2	Preprocess	Scale each series and transform it into fixed-length sliding windows and eight per-window statistical features.
FR-3	Train detectors	Train an Isolation Forest on features and an LSTM Autoencoder on normal-only windows, and persist both as artifacts.
FR-4	Score a window	Given a single window, return both models' scores and flags and the combined verdict.
FR-5	Score in batch	Score an entire series of windows in one request for visualisation.
FR-6	Combine models	Compute union and intersection of the two detectors' verdicts.
FR-7	Persist detections	Record every operational detection and the alert it triggers

		in a database.
FR-8	Alert	Send a Discord notification when an anomaly is detected, subject to a cooldown.
FR-9	Inject anomalies	Inject point, level-shift or trend-drift anomalies into a series on demand for demonstration.
FR-10	Evaluate	Compute precision, recall, F1, false-positive rate and precision–recall curves and serve them.
FR-11	Visualise	Display the metric, both anomaly signals, detected anomalies, history and metrics in a dashboard.

3.3 Non-Functional Requirements

The non-functional requirements describe qualities the system must exhibit rather than specific functions.

- **Performance:** model artifacts are loaded once at start-up so that per-request scoring incurs no loading cost; single-window scoring returns in well under a second.
- **Reliability:** persistence failures must never break inference — database writes are wrapped so that an anomaly is still reported even if logging fails.
- **Usability:** the dashboard must be operable by a non-programmer through dropdowns, radio buttons and sliders, with no code or query language required.
- **Reproducibility:** fixed random seeds and a single configuration file ensure that training and evaluation can be repeated to obtain the same artifacts and metrics.
- **Portability:** the system must run on a developer laptop and on free-tier cloud hosting without modification, using only open-source dependencies.
- **Maintainability:** the codebase must be modular and covered by automated tests so that components can be changed with confidence.
- **Cost:** the entire system must operate at zero licensing or infrastructure cost.

These requirements were maintained as a living checklist throughout development, and each is traceable to one or more components in the implementation and to one or more automated tests in Chapter 6. Treating the requirements as testable assertions rather than prose wishes was a deliberate methodological choice: a requirement that cannot be checked cannot be known to be met. The functional requirements above were prioritised so that the detection core (FR-1 to

FR-6) was built and validated first, with the surrounding service, persistence and presentation requirements layered on once the core was trustworthy.

The non-functional requirements were not treated as vague aspirations but as properties designed into the system and, where possible, observed in its behaviour. Performance is secured by loading models once at start-up; reliability by wrapping persistence so it cannot break inference; reproducibility by fixed seeds and a single configuration file; and portability by depending only on cross-platform open-source packages. The cost requirement of zero is satisfied absolutely, since every dependency is free and the system runs on existing hardware and free hosting tiers. Stating these qualities explicitly, and tying each to a concrete mechanism, ensured that they shaped the design rather than being asserted after the fact.

3.4 User Requirements

Two classes of user interact with Spike-Sense. The operations analyst is the primary user: they select a metric series, observe the detectors' output, investigate flagged windows, trigger demonstration anomalies, and review the alert and detection history. The evaluator — in the context of this academic project, the examiner — is a secondary user who needs to confirm that the system works, inspect the evaluation metrics, and understand the design. Both interact chiefly through the dashboard, with the API and its automatically generated documentation available for deeper inspection.

3.5 Use-Case Analysis

The use-case diagram in Figure 3.1 captures the goals each actor can accomplish with the system. The analyst can view the dashboard, score windows, inject demonstration spikes, view the detection history and retrain the models; the evaluator can view the dashboard, inspect evaluation metrics and run the injection demonstration; and the external Discord service receives alerts. The boundary of the system is drawn around the use cases it provides, with the actors outside it.

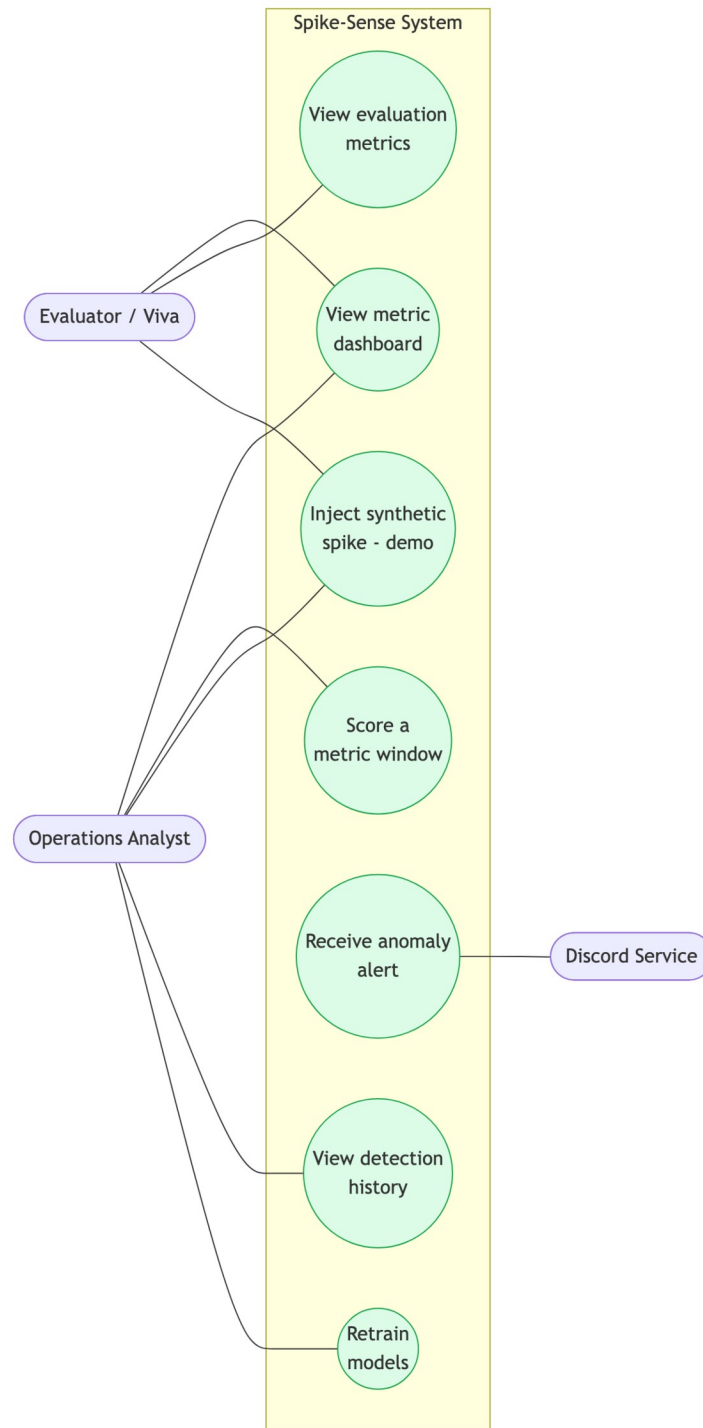


Figure 3.1: Use-case diagram showing the operations analyst and evaluator actors, the external Discord service, and the use cases provided within the Spike-Sense system boundary.

3.6 System Architecture (Analysis View)

At the analysis stage the architecture is best understood as a pipeline of responsibilities that data passes through, already introduced at a high level in Figure 1.1. Raw telemetry enters a data-handling responsibility that loads, labels, splits and transforms it; the transformed

windows pass to a modelling responsibility that learns normality and scores new windows; the scores pass to a decision responsibility that fuses them and decides whether to act; and the verdicts pass to interaction responsibilities that persist, alert and visualise. Each responsibility depends only on the output of the one before it, which is what allows them to be developed and tested in isolation. This conceptual architecture is refined into concrete layers and components in Chapter 4; here it serves to confirm that the requirements can be satisfied by a small number of cohesive, loosely coupled parts rather than a monolith, and that no requirement falls outside this structure.

3.7 Feasibility Study

3.7.1 Technical Feasibility

The project is technically feasible with widely available, well-documented open-source tools. Isolation Forest is provided by scikit-learn and the LSTM Autoencoder by Keras, both of which run on a standard CPU; no specialised hardware or GPU is required because the models are small and trained on at most a few tens of thousands of short windows. FastAPI, Streamlit and SQLite are similarly lightweight. The required skills — Python programming, machine learning and web development — are within the scope of the MCA curriculum.

3.7.2 Economic Feasibility

The project is economically feasible because it incurs no cost. All software is open source, the dataset is freely downloadable without registration, models are trained locally on an existing laptop, and deployment targets free hosting tiers. There are no licensing, data, compute or storage charges.

3.7.3 Operational Feasibility

The system is operationally feasible because it integrates with workflows operators already use. Alerts arrive in Discord, a chat platform widely used by engineering teams, and the dashboard runs in any web browser. No new client software needs to be installed, and the browser-based interface requires no training beyond familiarity with charts.

3.7.4 Schedule Feasibility

Finally, the project was feasible within the time available because its modular structure allowed it to be built and validated in independent increments. The data pipeline, the two models, the evaluation harness, the service layer, the persistence layer and the dashboard each constituted

a self-contained unit of work that could be completed and tested before the next began. This decomposition reduced risk: a difficulty in one module — for instance tuning the autoencoder threshold — could be resolved without blocking progress on others, and the committed, pre-trained model artifacts meant that later work on the API and dashboard never depended on retraining.

3.8 Data-Flow Diagrams

Data-flow diagrams model the movement and transformation of data through the system. The context-level diagram, Figure 3.2, treats the whole system as a single process exchanging data with three external entities: the NAB dataset, which supplies labelled metric series; the operations analyst, who issues requests and receives verdicts and metrics; and the Discord service, which receives alerts.

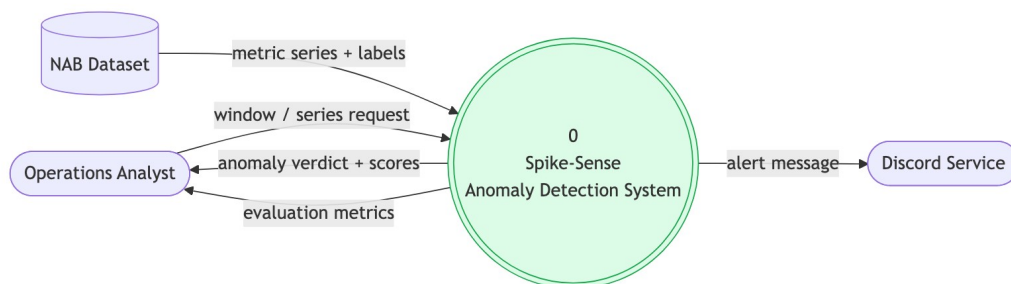


Figure 3.2: Level-0 (context) data-flow diagram. The system is a single process exchanging data with the NAB dataset, the analyst, and the Discord service.

The level-1 diagram, Figure 3.3, decomposes the system into its six principal processes — loading and labelling, preprocessing, scoring, combination and decision, alerting and persistence, and evaluation — and shows the data stores between them: the processed window arrays, the persisted model artifacts, the SQLite detection database and the JSON evaluation results. This decomposition mirrors the module structure of the implementation.

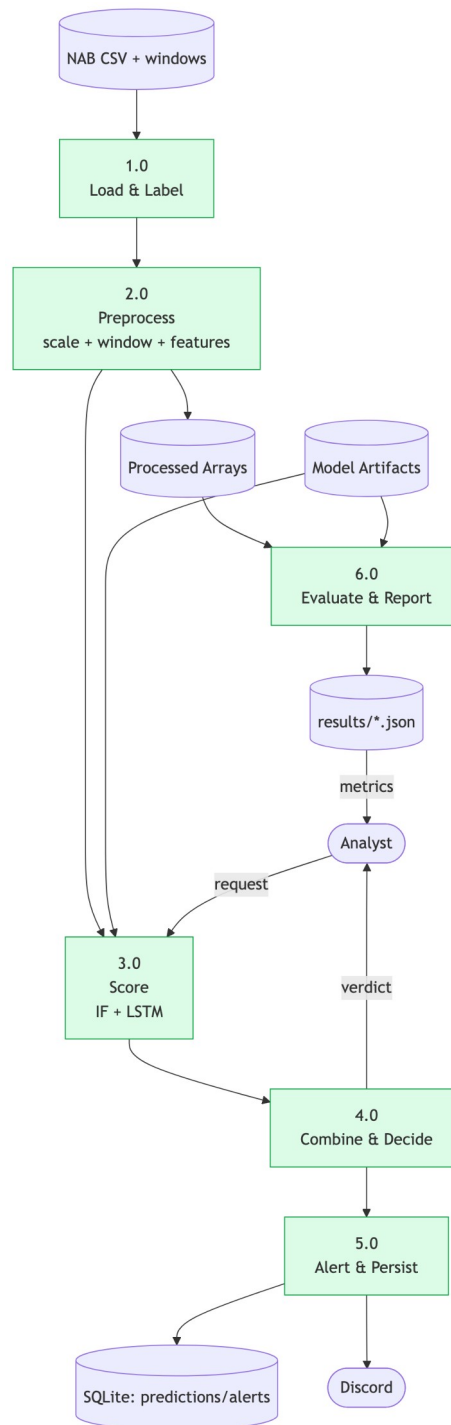


Figure 3.3: Level-1 data-flow diagram decomposing the system into its six processes and four data stores.

Reading the two diagrams together clarifies an important property of the system: the heavy, offline transformations and the light, online ones are cleanly separated. Loading, preprocessing and model training (processes one and two, feeding the model-artifact store) happen ahead of time and produce durable artifacts; scoring, decision, alerting and evaluation (processes three to six) operate at request time against those artifacts. This separation, visible in the data stores

that sit between the processes, is what allows the online path to be fast and the offline path to be thorough, and it foreshadows the layered implementation described in Chapter 5.

3.9 Summary

The analysis confirms that Spike-Sense is feasible on every axis and establishes a clear set of functional and non-functional requirements together with validated models of its users and data flows. The functional requirements pin down what the system must do, the non-functional requirements pin down how well, the use cases capture who needs what, and the data-flow diagrams capture how information moves and is stored. Each of these will be seen again in concrete form in the design and implementation chapters, where the abstract processes become Python modules and the abstract data stores become files, model artifacts and database tables. These requirements and models drive the detailed design presented in the next chapter.

Chapter 4: System Design

4.1 Introduction

This chapter presents the design of Spike-Sense at the level of detail needed to implement it. It describes the layered architecture, the principal classes and their relationships, the dynamic behaviour of the system through sequence and activity diagrams, the relational database schema with its entity–relationship model, and the design of the REST API. Each diagram is numbered and explained, and the rationale behind significant design decisions is made explicit.

4.2 Architectural Design

Spike-Sense follows a layered architecture, introduced in Figure 1.1, comprising four layers. The data layer loads, labels, splits and preprocesses telemetry. The model layer trains and holds the two detectors. The service layer — a FastAPI application — loads the trained artifacts once at start-up into a singleton registry and exposes them through REST endpoints, fanning out to the alerting and persistence components. The presentation layer is the Streamlit dashboard, which communicates with the service layer purely over HTTP and holds no model logic of its own. This separation means the computationally heavy model code and the lightweight presentation code can be deployed independently — the API on a service host and the dashboard on a separate free tier — and that the dashboard’s dependencies remain minimal.

Three design principles guided the architecture. The first is separation of concerns: each layer has one responsibility and communicates with its neighbours through a narrow, well-defined interface — the data layer emits arrays, the model layer emits scores, the service layer emits JSON, and the presentation layer consumes JSON. The second is configuration over hard-coding: every tunable quantity, from the list of series to the model hyper-parameters, lives in a single configuration file, so the behaviour of the whole pipeline can be changed and reproduced without editing code. The third is fail-safe degradation: auxiliary concerns such as persistence and alerting are wrapped so that their failure can never prevent the primary task of producing a detection. Together these principles keep the system understandable, reproducible and robust.

The decision to load all model artifacts once into a singleton registry at start-up, rather than per request, is an example of these principles applied for performance. Deserialising a Keras model and refitting per-series scalers is comparatively expensive; doing it once at start-up means that the hot path — scoring a window — touches only in-memory objects and returns in well under

a second. The registry also localises the knowledge of where artifacts live and how they are loaded, so the endpoints themselves remain ignorant of those details.

The layering also defines the deployment topology. The service and presentation layers are designed to run as two independent processes that need not share a host: the FastAPI service can be deployed on a small application host, while the Streamlit dashboard can run on a separate free tier and reach the service over HTTP using a configurable base URL. Because the dashboard holds no model code, its deployment is lightweight and its dependency set small. The trained model artifacts are committed to the repository, so the service can start and serve immediately on a fresh host without a training step, and the SQLite database is a single file created on first run. This topology was chosen specifically so that the whole system fits within free hosting tiers, honouring the zero-cost constraint while remaining a realistic two-tier web architecture.

4.3 Class Design

Although much of the codebase is organised as cohesive modules of functions, several classes and structured records form the backbone of the system. Figure 4.1 shows the principal ones. The `ModelRegistry` is a singleton that owns the loaded Isolation Forest, the loaded LSTM, the decision threshold, the per-series scalers and the window size, and exposes them to the API. `PreprocessedData` is an immutable record bundling the outputs of preprocessing — the two-dimensional windows, the three-dimensional windows for the LSTM, the feature matrix, the labels and the timestamps. `SeriesSplit` holds the chronological train, validation and test partitions of a series. `AlertState` tracks the cooldown that prevents alert flooding. `Prediction` and `Alert` are the two persisted entities, related one-to-many.

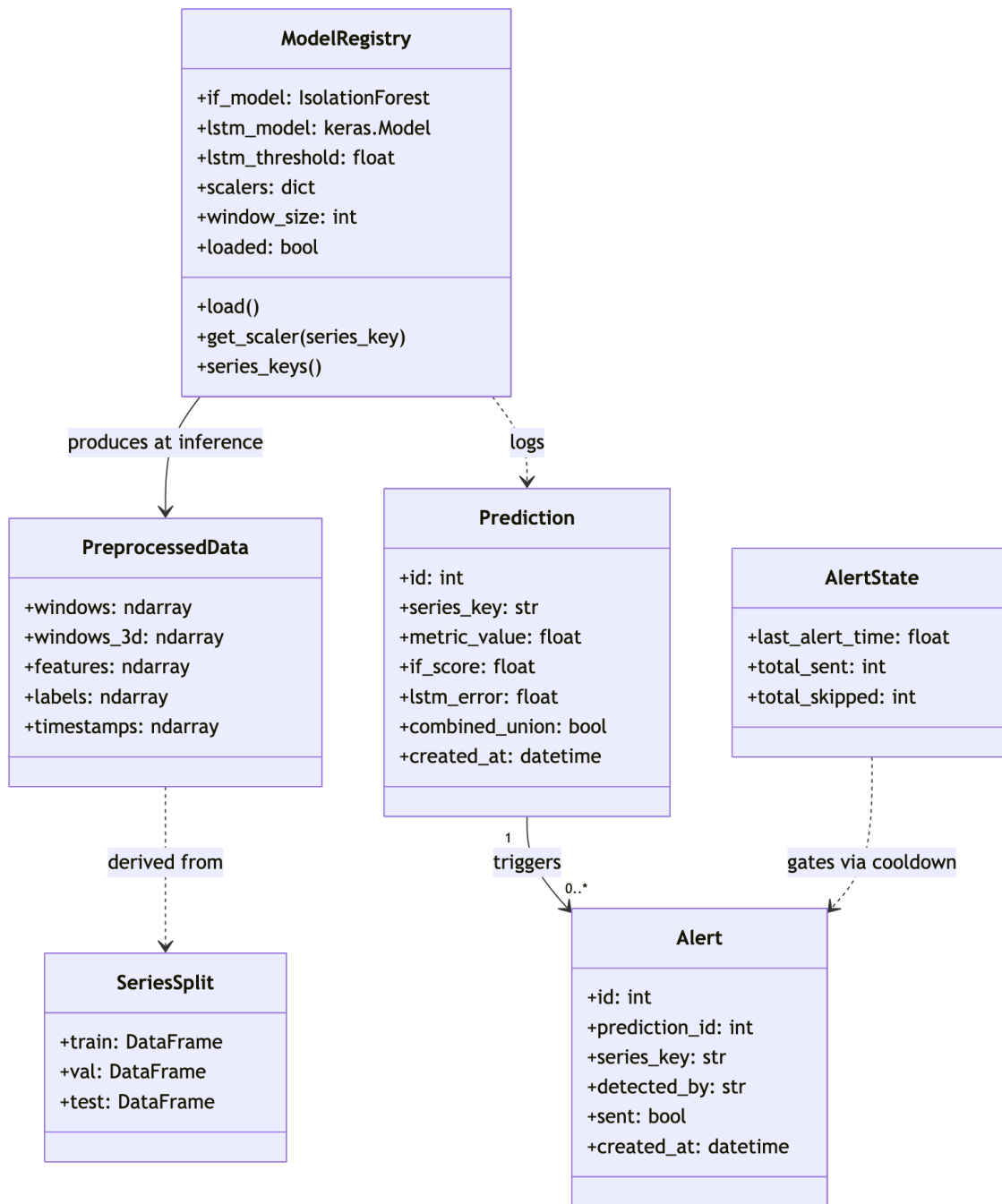


Figure 4.1: Class diagram of the principal classes and structured records, including the model registry, the preprocessing and split records, the alert-state tracker, and the two persisted entities.

The relationships among these elements are mostly compositional and procedural rather than inheritance-based, which suits the data-processing nature of the system. The registry produces, at inference time, the same windowed representations that the preprocessing record describes; the preprocessing record is itself derived from a chronological split; the two persisted entities stand in a one-to-many relationship; and the alert-state tracker gates whether a detection

becomes a notification. Keeping these as small, single-purpose types with clear ownership of their data makes the flow of information through the system easy to follow and to test.

4.4 Database Design

Spike-Sense persists its runtime history in a relational SQLite database accessed through the SQLAlchemy object–relational mapper. SQLite was chosen because it is serverless, file-based, requires no separate installation, commits to the repository, and runs identically on a laptop and on free hosting — matching the project’s zero-cost and portability requirements. The schema comprises two tables in a one-to-many relationship: a prediction may give rise to zero or more alerts, because the cooldown can suppress the notification for a detection.

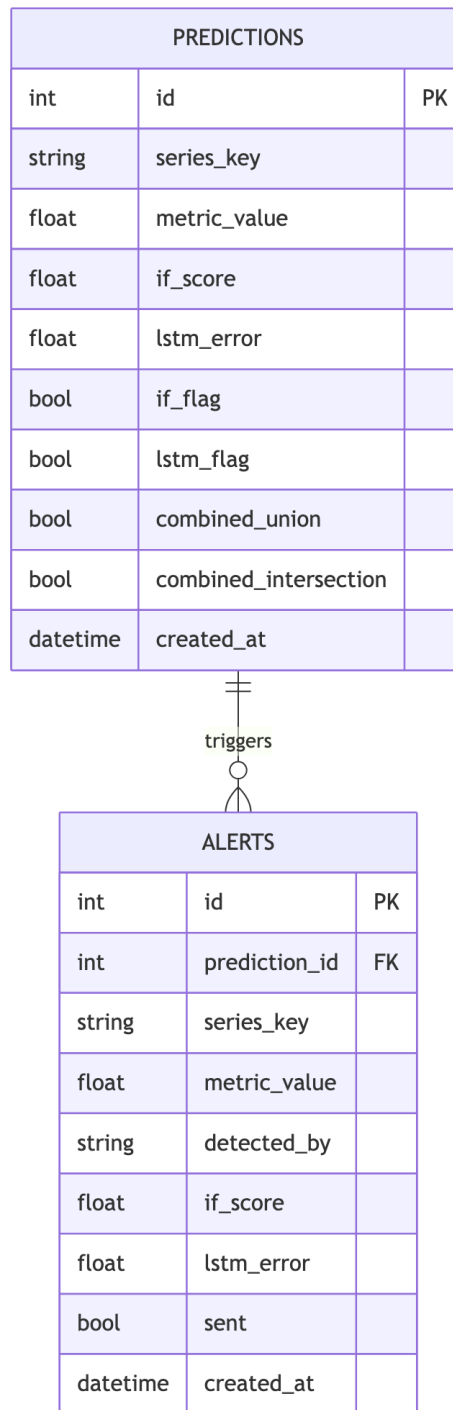


Figure 4.2: Entity–relationship diagram of the persistence layer. The *PREDICTIONS* entity is related one-to-many to the *ALERTS* entity through the *prediction_id* foreign key.

The predictions table records each operational detection: the series it came from, the metric value, both models’ scores, the individual and combined verdicts, and a timestamp. The alerts table records each notification that a detection produced, linked back to its prediction by a foreign key, together with which models fired, whether delivery to Discord succeeded, and a timestamp. Table 4.1 details the columns of both tables.

Table 4.1: Database schema: columns of the predictions and alerts tables.

Table	Column	Type	Description
predictions	id	INTEGER PK	Auto-increment primary key.
predictions	series_key	TEXT	NAB series identifier (indexed).
predictions	metric_value	REAL	Last raw value of the scored window.
predictions	if_score	REAL	Isolation Forest anomaly score.
predictions	lstm_error	REAL	LSTM reconstruction error.
predictions	if_flag / lstm_flag	BOOLEAN	Per-model anomaly verdicts.
predictions	combined_union / _intersection	BOOLEAN	Fused verdicts.
predictions	created_at	DATETIME	Detection timestamp (indexed).
alerts	id	INTEGER PK	Auto-increment primary key.
alerts	prediction_id	INTEGER FK	References predictions.id.
alerts	series_key	TEXT	Series identifier (indexed).
alerts	metric_value	REAL	Value that triggered the alert.
alerts	detected_by	TEXT	Which models fired: IF, LSTM or IF+LSTM.
alerts	if_score / lstm_error	REAL	Scores at the time of alert.
alerts	sent	BOOLEAN	Whether Discord delivery succeeded.
alerts	created_at	DATETIME	Alert timestamp (indexed).

A note on the modelling style is warranted. Spike-Sense is not a heavily object-oriented system; much of its logic lives in cohesive modules of functions — load, preprocess, train, score,

evaluate — because the data-pipeline and machine-learning code is naturally expressed as transformations of arrays rather than as collaborating objects. Classes are introduced only where they earn their place: the registry, because it holds shared mutable state that must be initialised once and accessed everywhere; the structured records, because they bundle related arrays that travel together through the pipeline; and the two persisted entities, because the object–relational mapper expresses tables as classes. This pragmatic mix of functional and object-oriented style keeps each part of the system in its most natural form.

4.5 API Design

The service layer exposes a REST API whose endpoints map directly onto the functional requirements. Each request and response is validated against a Pydantic schema, and FastAPI generates interactive documentation automatically. Table 4.2 summarises the endpoints.

Table 4.2: *REST API endpoints exposed by the FastAPI service layer.*

Method	Path	Purpose
GET	/health	Liveness check; reports whether models are loaded.
GET	/info	Model metadata: window size, thresholds, available series.
POST	/predict	Score a single window; persists and alerts on detection.
POST	/predict/batch	Score many windows for visualisation (read-only).
GET	/evaluate	Return pre-computed evaluation metrics and scenarios.
POST	/demo/inject-spike	Inject a synthetic anomaly and score the series.
GET	/alerts	Return recent persisted alerts from the database.
GET	/stats	Return aggregate detection and alert counts.

Several decisions shaped the API design. The endpoints are split into read operations, which use GET and are free of side effects, and scoring operations, which use POST. A deliberate and important distinction is drawn between single-window scoring, which is treated as an operational event that persists a detection and may raise an alert, and batch scoring, which is

treated as a read-only visualisation scan that neither persists nor alerts. Without this distinction, a single dashboard load — which scores a whole series in batch — would write thousands of rows into the detection history and could trigger a storm of notifications; separating the two intents keeps the persisted history meaningful. Input is validated declaratively: a window of the wrong length, or a request parameter outside its permitted range, is rejected before any model is invoked, with an informative error and an appropriate status code. This validation is generated automatically from the typed schemas, so the documentation and the enforced contract can never drift apart.

4.6 Dynamic Behaviour

4.6.1 Single-Window Scoring Sequence

Figure 4.2 traces the flow of a single scoring request. The dashboard sends the raw window to the API, which obtains the appropriate per-series scaler from the registry, scales the window, extracts features for the Isolation Forest and shapes the window for the LSTM, invokes both models, combines their verdicts, and — if an anomaly is detected — persists the detection, fires an alert subject to cooldown, and records the alert. The response carries both flags, both scores and the alert status back to the dashboard for display.

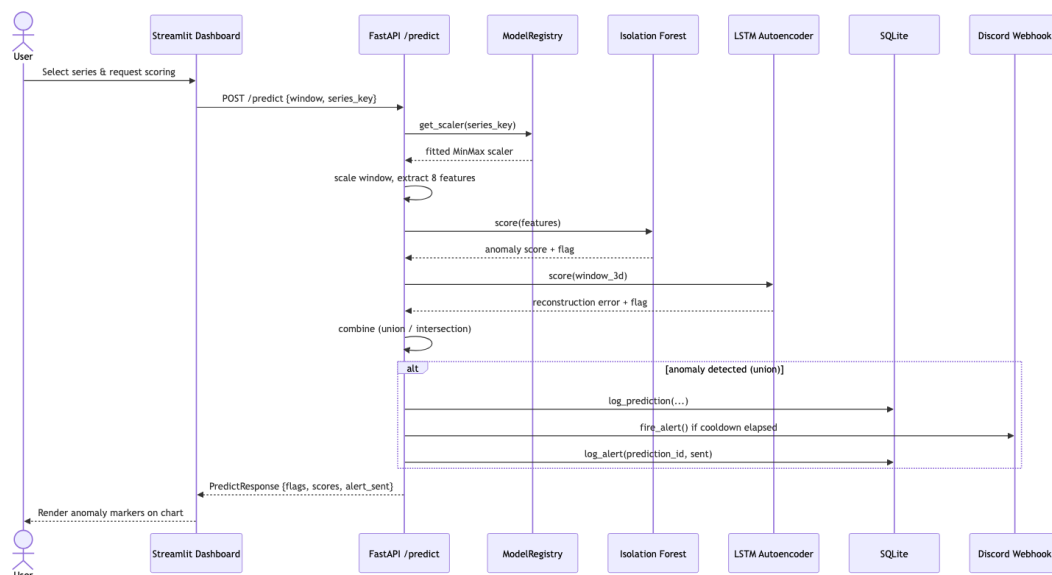


Figure 4.3: Sequence diagram of single-window scoring, from the dashboard request through scaling, both models, combination, persistence and alerting, to the response.

The sequence makes explicit a property that is easy to overlook: the scaling step uses the scaler belonging to the named series, not a generic one. This is why the request optionally carries a series key — supplying it lets the service normalise the window in the correct context, which

materially affects the scores. The diagram also shows persistence and alerting as a conditional branch taken only when an anomaly is detected and only on the operational scoring path, so that the common case of a normal window returns immediately with no side effects.

4.6.2 Spike-Injection Sequence

Figure 4.3 traces the demonstration flow. The dashboard requests an injection of a chosen type and magnitude; the API takes a clean copy of the series, injects the synthetic anomaly at its midpoint, preprocesses and batch-scores the whole series, persists the first detection, and returns the per-window verdicts so the dashboard can show the injected anomaly being caught.

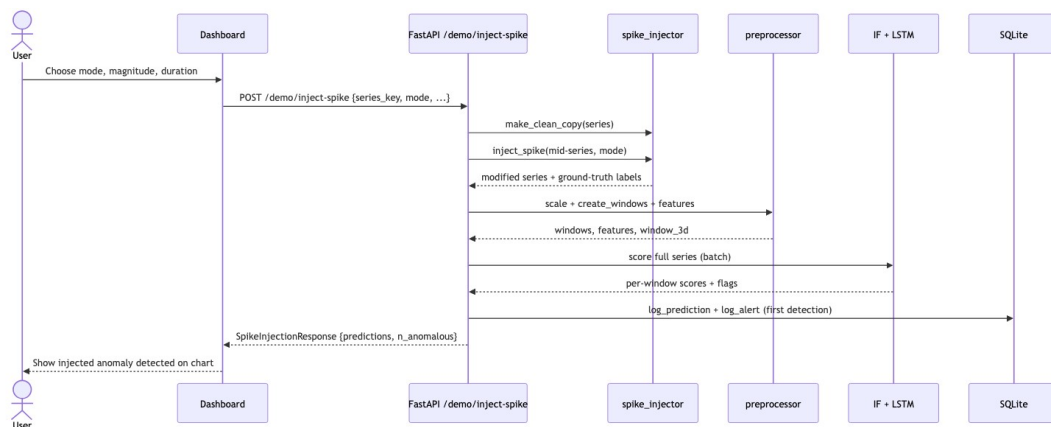


Figure 4.4: Sequence diagram of the spike-injection demonstration, showing injection, preprocessing, batch scoring and persistence of the triggering detection.

4.6.3 Training Activity

Figure 4.4 is an activity diagram of the offline training pipeline. For each series it performs a chronological split, fits a scaler on the training partition only, windows and extracts features, and saves the processed arrays. It then aggregates windows across series, sweeps and trains the Isolation Forest, filters to normal-only windows to train the LSTM Autoencoder with early stopping, computes the reconstruction-error threshold, and saves both artifacts.

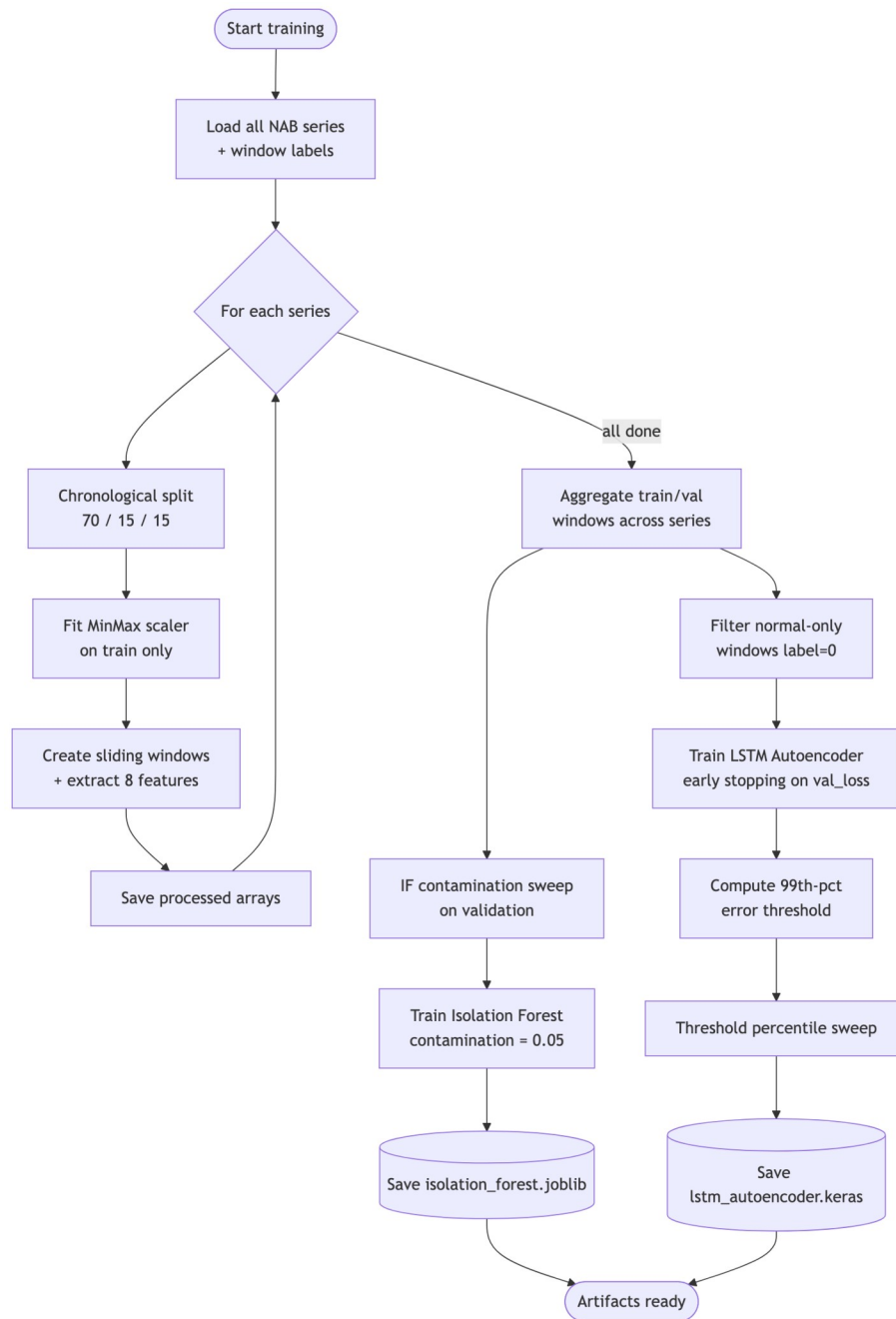


Figure 4.5: Activity diagram of the training pipeline, from per-series preprocessing through aggregation to the training and persistence of both models.

The alerts table intentionally duplicates a few fields from its parent prediction — the series, the value and the scores. In a strictly normalised schema these would be reached through the foreign key, but denormalising them onto the alert makes the alert self-describing: a query of recent alerts for the dashboard or the history endpoint returns everything needed to render them without a join. Given the small scale and read-mostly nature of the history, this is a sound trade of a little redundancy for simpler, faster reads. The created_at timestamps on both tables are

indexed because the most common query — ‘show me the most recent detections and alerts’ — orders by time.

4.7 Design of the Detection Models

The Isolation Forest is configured with two hundred trees and a contamination parameter of 0.05, operating on eight features computed per window: mean, standard deviation, minimum, maximum, range, root-mean-square, skewness and kurtosis. These descriptors capture the level, spread, extremes and shape of each window in a compact, interpretable vector. The LSTM Autoencoder uses an encoder LSTM of sixty-four units, a thirty-two-unit dense bottleneck, a sixty-four-unit decoder and a time-distributed output layer, with light dropout for regularisation. It is trained with the Adam optimiser to minimise mean-squared reconstruction error on normal windows only, with early stopping on validation loss. The anomaly threshold is set at the ninety-ninth percentile of the training reconstruction errors, a conservative choice favouring precision that can be retuned from the validation sweep presented in Chapter 7.

The window size of thirty steps — two and a half hours at the five-minute sampling rate — was chosen as a balance between context and responsiveness. Too short a window would deprive the autoencoder of the temporal context it needs to recognise shape, and would make the statistical features noisy; too long a window would blur brief anomalies into a mostly normal background and delay detection until the window filled. Thirty steps is long enough to capture the local trend and variability of a metric yet short enough that a developing fault raises the signal within a reasonable time. The stride of one step means windows overlap almost completely, so every new sample produces a fresh verdict and no anomaly can slip between non-overlapping windows; the cost is more windows to score, which the lightweight models absorb easily.

4.8 Summary

The design establishes a clean layered architecture, a small set of well-defined classes, a normalised two-table database, a REST API aligned with the requirements, and explicit dynamic models of the system’s key flows. Every requirement from Chapter 3 maps onto a concrete element here — functions in the data and model layers, endpoints in the service layer, tables in the database, and panels in the dashboard — and every figure has been numbered and explained so that the design can be implemented without ambiguity. With the design fixed, the next chapter turns to its implementation.

Chapter 5: System Implementation

5.1 Introduction

This chapter describes how the design of Chapter 4 was realised in code. It sets out the development environment and the tools used, then walks through the implementation module by module — data pipeline, models, evaluation, service layer, persistence and dashboard — explaining the logic of each and illustrating it with short, representative code excerpts rather than complete listings, which are provided in the appendices. Screenshots of the running application demonstrate the implemented behaviour.

5.2 Development Environment

Development was carried out in Python 3.10 on macOS, in an isolated virtual environment so that dependencies were pinned and reproducible. Visual Studio Code served as the editor, Git as the version-control system, and pytest as the test runner. The same code runs unchanged on Linux, which is the target for deployment on free hosting tiers.

Dependencies are pinned to specific versions in a requirements file, and a lighter, separate requirements file is maintained for the dashboard so that its deployment need not pull in the heavy machine-learning stack — the dashboard communicates with the API over HTTP and therefore requires neither TensorFlow nor the model libraries. This two-file arrangement reflects the architectural separation in the dependency graph itself, keeping each deployment target as small as it can be. Pinning versions, rather than accepting whatever is latest, is what makes the build reproducible months later and on a different machine, which is essential for a project intended to be re-run and re-evaluated.

5.3 Tools and Technologies Used

Table 5.1: *Tools and technologies used in the implementation, with versions and roles.*

Layer	Technology	Version	Role
Language	Python	3.10	Primary implementation language.
ML — classical	scikit-learn	1.7.2	Isolation Forest detector.
ML — deep	TensorFlow / Keras	2.16.1	LSTM Autoencoder.
Data	NumPy / pandas / SciPy	1.26 / 2.3 / 1.15	Arrays, frames, statistics.
API	FastAPI / Uvicorn	0.136 / 0.45	REST service and ASGI server.

Validation	Pydantic	2.13	Request/response schemas.
Persistence	SQLAlchemy / SQLite	2.0.50 / built-in	Detection and alert history.
Dashboard	Streamlit / Plotly	1.35 / 5.22	Interactive visualisation.
Alerting	requests + Discord webhook	2.33	Real-time notifications.
Testing	pytest / pytest-cov	9.0 / 7.1	Automated tests and coverage.

5.4 Hardware and Software Requirements

The system has modest requirements. Training and serving run comfortably on a standard laptop with a multi-core processor and eight gigabytes of memory; no GPU is needed. The only software prerequisites are a Python 3.10-or-later interpreter and the pinned open-source packages listed above, all installable with a single command from the requirements file.

The implementation is organised into packages that mirror the layered architecture: a data package, a models package, an evaluation package, an api package and a dashboard package, with thin orchestration scripts for training and evaluation. This correspondence between the design and the directory structure is intentional, because it makes the codebase navigable: a reader who has understood the architecture already knows where to find any piece of functionality. The following subsections walk through each package in the order data flows through the system, from raw telemetry to served prediction.

5.5 Module-Wise Implementation

5.5.1 Data Loading and Labelling

The loader reads each NAB CSV into a pandas frame, parses timestamps, drops missing values, and attaches a binary label. Following NAB's official scoring convention, it labels not single instants but every sample that falls inside a labelled anomaly window. The core of the labelling logic is shown below.

Listing 5.1 — *Window-based anomaly labelling in loader.py.*

```
for entry in entries:
    if isinstance(entry, (list, tuple)) and len(entry) == 2:
        start, end = pd.Timestamp(entry[0]), pd.Timestamp(entry[1])
        label |= ((df['timestamp'] >= start) &
                  (df['timestamp'] <= end)).astype(int)
```

```
else: # backward-compatible single-point label
    label |= (df['timestamp'] == pd.Timestamp(entry)).astype(int)
df[label_col] = label.astype(int)
```

The choice to label by window rather than by instant is significant and was a refinement made during development. NAB publishes both single-point labels and anomaly windows; the windows are the benchmark's official scoring labels and reflect the operational truth that a fault occupies an interval, not a single five-minute sample. Labelling only the exact instant would leave barely a handful of positive samples per series — too few to evaluate meaningfully and unrepresentative of how the fault actually appears in the data. Adopting the windows raised the positive rate to a realistic eight to ten percent and, by selecting series whose windows fall late in the timeline, ensured that genuine anomalies survive into the chronological test partition. The loader retains backward compatibility with the point-label format so that either labelling scheme can be used by changing one configuration entry.

The loader also enforces basic data hygiene before any modelling begins. Timestamps are parsed and the series is sorted chronologically so that windowing reflects true temporal order; rows with missing values are dropped with a logged warning rather than silently, so that data-quality issues are visible; and the file key that links a series to its labels is inferred from the path when not supplied, which keeps the configuration concise. These steps are unglamorous but essential — a detector is only as trustworthy as the data feeding it, and a mis-parsed timestamp or an unnoticed gap would quietly corrupt every downstream result.

5.5.2 Preprocessing and Feature Extraction

The preprocessor fits a min-max scaler on the training partition only — never on validation or test data — to avoid leakage, then slides a fixed-length window across the scaled series. For the Isolation Forest it computes eight statistical features per window; for the LSTM it retains the raw scaled window reshaped to three dimensions. The feature extraction is summarised below.

Listing 5.2 — Per-window statistical feature extraction.

```
means = windows.mean(axis=1); stds = windows.std(axis=1)
mins = windows.min(axis=1); maxs = windows.max(axis=1)
ranges = maxs - mins
rms = np.sqrt((windows ** 2).mean(axis=1))
skews = np.array([skew(w) for w in windows])
kurts = np.array([kurtosis(w) for w in windows])
features = np.column_stack([means, stds, mins, maxs,
                             ranges, rms, skews, kurts])
```

The eight features were chosen to summarise a window from complementary angles. Mean and root-mean-square capture its central level; standard deviation and range capture its dispersion;

minimum and maximum capture its extremes; and skewness and kurtosis capture the shape of its distribution — asymmetry and tailedness — which distinguish, for example, a window containing a single sharp spike from one that is merely noisy. Reducing a thirty-step window to these eight numbers is a deliberate trade: it discards temporal order, which is why the Isolation Forest is paired with the order-aware autoencoder, but in exchange it gives the forest a compact, scale-aware and interpretable description on which it can isolate outliers efficiently. Fitting the scaler on the training partition alone, and then applying that same fitted scaler to validation and test data, is the single most important guard against data leakage in the pipeline; scaling the whole series at once would let information about future extremes influence the normalisation of the past.

5.5.3 The Detection Models

The Isolation Forest module wraps scikit-learn, exposing train, score, predict and a contamination sweep. Scores negate the library's decision function so that higher means more anomalous. The LSTM module builds the encoder–bottleneck–decoder network in Keras, trains it on normal windows, computes per-window reconstruction error as mean-squared error, and derives the anomaly threshold as a percentile of training errors.

Listing 5.3 — LSTM Autoencoder architecture (Keras).

```
model = Sequential([
    LSTM(encoder_units, activation='tanh',
         input_shape=(window_size, 1)),
    RepeatVector(window_size),
    LSTM(decoder_units, activation='tanh', return_sequences=True),
    Dropout(dropout),
    TimeDistributed(Dense(1)),
])
model.compile(optimizer=Adam(learning_rate), loss='mse')
```

Two implementation details of the autoencoder deserve emphasis. First, training uses only the windows whose label is zero — the normal windows — even though the labels are never used as a learning target. This is what makes the approach semi-supervised: the network learns the manifold of normal behaviour and is never shown how to reconstruct an anomaly, so anomalies remain hard for it to reproduce. Second, training is governed by early stopping on the validation loss with a small patience, so the number of epochs is determined by the data rather than fixed in advance; in practice the network converges within a handful of epochs because normal cloud telemetry is highly regular. After training, the anomaly threshold is fixed at a high percentile of the reconstruction errors observed on the training data, a conservative setting that keeps the

false-positive rate low and that can be relaxed using the validation sweep presented in Chapter 7 when higher recall is desired.

5.5.3a The Model Registry and Per-Series Scaling

A subtle but important part of the implementation is the model registry, the singleton that holds the loaded artifacts for the lifetime of the service. At start-up it deserialises the Isolation Forest and the LSTM, reads their metadata — the decision threshold, the window size, the error statistics — and rebuilds the per-series scalers by refitting them on each series' training partition. The scalers are deliberately not serialised but reconstructed, which keeps the committed artifacts small and guarantees that a scaler always matches the current data and split definition. Holding all of this in one place means the endpoints can remain ignorant of how artifacts are stored and loaded; they simply ask the registry for the scaler belonging to a series and for the models, and the registry guarantees they are ready.

Per-series scaling is essential to correctness. The six series live on incomparable scales — processor percentages bounded at a hundred, network byte counts in the millions, latencies in milliseconds — so a single global scaler would compress the small-scale series into a sliver of the range and destroy the very deviations the detectors must see. By fitting and applying a separate scaler per series, each metric is normalised within its own context, and the registry exposes the correct scaler for whichever series a request names, falling back to a sensible default when none is given. This design lets one pair of models serve heterogeneous metrics without retraining per series.

5.5.4 Ensemble Combination and Scoring

The service layer's scoring helper ties the models together. It scales the incoming window with the correct per-series scaler, derives both representations, queries both models, and combines their verdicts by union and intersection. When an operational detection occurs it persists the record, fires an alert under the cooldown policy, and persists the alert; batch scoring for the dashboard runs the same logic in a read-only mode that neither alerts nor persists, so that bulk visualisation cannot flood the history.

Listing 5.4 — Ensemble combination, alerting and persistence in main.py.

```
combined_union = if_pred or lstm_pred
combined_intersection = if_pred and lstm_pred
if combined_union and persist:
    alert_sent = fire_alert(...)
    pred_id = db.log_prediction(...)
    db.log_alert(prediction_id=pred_id, sent=alert_sent, ...)
```

5.5.5 Persistence Layer

The database module defines the two ORM entities with SQLAlchemy's declarative mapping and provides helper functions to log predictions and alerts and to query recent history and aggregate statistics. The one-to-many relationship is expressed through a foreign key and a relationship attribute, and the engine is configured so that the FastAPI thread pool can share the SQLite file safely.

Listing 5.5 — Declarative ORM model for the predictions table.

```
class Prediction(Base):
    __tablename__ = 'predictions'
    id: Mapped[int] = mapped_column(primary_key=True)
    series_key: Mapped[str | None] = mapped_column(String(128), index=True)
    combined_union: Mapped[bool] = mapped_column(Boolean)
    alerts: Mapped[list['Alert']] = relationship(back_populates='prediction')
```

Two further design points make the persistence safe and meaningful. First, every write is wrapped so that a database error is caught and logged but never propagates into the scoring path: the cardinal rule is that a failure to record a detection must not prevent the detection itself from being reported. Second, the engine is configured to allow the FastAPI worker threads to share the single SQLite file safely, and the schema is created idempotently at start-up, so the system is ready to persist from the first request without a separate migration step. For a single-node, read-mostly history of modest volume, SQLite is not a compromise but the right tool — it removes an entire class of operational burden, needing no server to run, secure or back up beyond copying a file.

5.5.6 Alerting

The alerting module formats a human-readable message naming the series, the value and the models that fired, and posts it to a Discord webhook whose URL is read from an environment variable. A cooldown prevents flooding: if an alert was sent within the configured interval the new one is skipped. When no webhook is configured the message is logged instead, so the system runs safely in local development.

The cooldown deserves a word, because alert fatigue is one of the chief reasons operators come to ignore monitoring systems. A genuine incident often produces a run of consecutive anomalous windows; without suppression, each would fire its own notification and bury the signal under repetition. The cooldown collapses such a burst into a single timely alert while still recording every underlying detection in the database, so that no information is lost even though the operator is notified only once. This separation of what is persisted from what is

pushed is a small but important piece of operational design. Discord was chosen as the delivery channel because its incoming-webhook mechanism is free, requires no application to be registered, and reaches a platform engineering teams already watch, keeping faithful to the project's zero-cost constraint.

A small but important reliability detail is that the database engine is created lazily and the schema is created idempotently, so the very first request finds the tables ready and concurrent requests from the server's worker threads can share the single SQLite file safely. The read helpers return plain dictionaries rather than live ORM objects, which keeps the database session short-lived and the API layer decoupled from the persistence layer's internal types — the endpoints serialise these dictionaries directly to JSON. This clean boundary means the storage technology could later be swapped for a client-server database without changing a single line in the endpoints.

5.5.7 REST API

The FastAPI application loads the models and initialises the database in a start-up lifespan handler, then serves the eight endpoints of Table 4.2. Its automatically generated documentation, shown in Figure 5.1, lists every endpoint grouped by tag along with the schemas, and lets a user execute requests interactively from the browser.

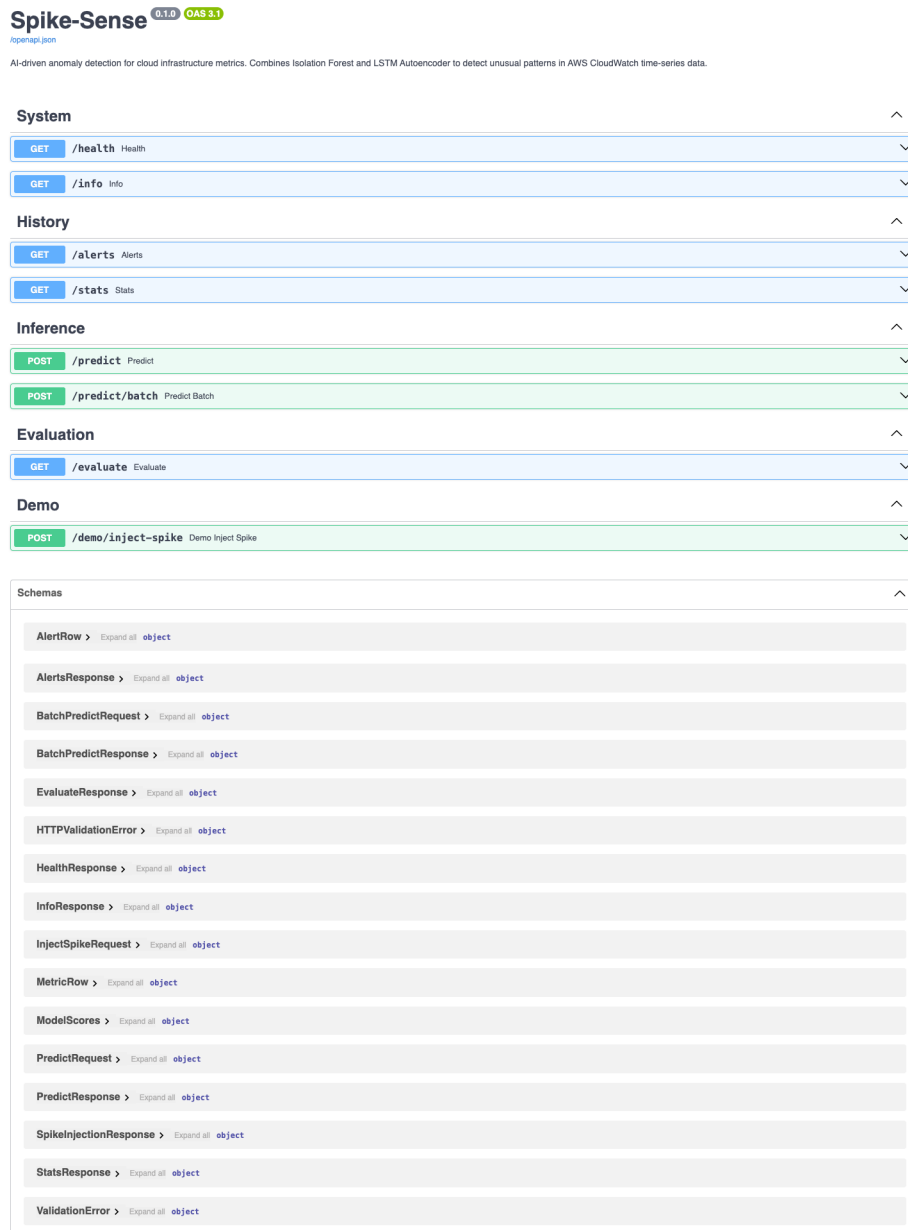


Figure 5.1: Automatically generated Swagger (OpenAPI) documentation of the Spike-Sense REST API, showing the System, History, Inference, Evaluation and Demo endpoint groups and the data schemas.

FastAPI was selected for the service layer for three reasons. It derives request validation and the interactive documentation directly from Python type annotations, so the contract, the validation and the documentation are a single source of truth that cannot fall out of step. It is built on an asynchronous server, so it remains responsive under concurrent requests. And it integrates cleanly with Pydantic, whose models express each endpoint's input and output as typed classes. The models are loaded once in a start-up lifespan handler and held in the registry for the life of the process, and the database is initialised in the same handler, so the first request is served against a fully warm system. The interactive documentation visible in the figure is

not an afterthought but the canonical interface through which the API can be explored and exercised, which is particularly useful for an examiner wishing to confirm the system's behaviour directly.

5.5.8 Dashboard

The Streamlit dashboard is the operator's window onto the system. A sidebar selects the series, chooses which models' anomalies to display, and configures and triggers spike injection. The main panel shows a three-row Plotly chart: the metric value with ground-truth and detected anomalies marked, the Isolation Forest score, and the LSTM reconstruction error. A collapsible panel shows the evaluation metrics. The dashboard talks to the API only over HTTP, so it carries none of the model dependencies itself.

Streamlit was chosen over a hand-built front end because it lets a data application be expressed in pure Python, with widgets and charts declared as ordinary statements and the framework handling the browser, the event loop and the re-rendering. This kept the presentation layer to a few hundred lines while still delivering an interactive, professional interface. The three-row layout is a considered design: stacking the raw metric above the two detectors' signals, on a shared time axis, lets an operator see at a glance not only that an anomaly was flagged but why — whether it was the Isolation Forest's score or the autoencoder's reconstruction error, or both, that crossed its threshold at that moment. This visual juxtaposition is the dashboard's main contribution to explainability, turning two opaque model outputs into a legible, comparable story.

Performance was a real consideration in the dashboard, because scoring a full series means sending every window to the API. The application caches loaded series and scored results in the session so that switching display options does not trigger needless rescoring, and it uses the batch endpoint so that an entire series is scored in a single request rather than thousands. This keeps the interface responsive even on the longest series.



Figure 5.2: The Spike-Sense dashboard displaying a real EC2 CPU series. The top panel shows the metric with ground-truth anomalies; the lower panels show the two detectors' signals.

Figure 5.3 shows the dashboard after a synthetic point spike has been injected. The injected anomaly is caught by the detectors — visible as the detected-anomaly markers clustered at the injection — and the summary line reports the window counts and detections, while the reconstruction-error panel shows a clear pair of peaks where the autoencoder fails to reproduce the anomalous region.



Figure 5.3: The dashboard after injecting a point spike. The detectors flag the injected region; the lower panels show the corresponding rise in Isolation Forest score and LSTM reconstruction error.

Detections made through the operational endpoint are persisted and can be retrieved through the history API. Figure 5.4 shows the JSON returned by the alerts endpoint, including which models fired and whether the Discord delivery succeeded.

```
pretty print
[{"alerts": [{"id": 4, "prediction_id": 4, "series_key": "realtimeCloudwatch/ec2_cpu_utilization_825cc2.csv", "metric_value": 95.876, "detected_by": "IF", "if_score": 0.01376873458177988, "lstm_error": 0.001552858822918117, "sent": false, "created_at": "2026-05-30T13:25:28.484151Z"}, {"id": 13, "prediction_id": 13, "series_key": "realtimeCause/cpu_utilization_avg_misconfiguration.csv", "metric_value": 31.965, "detected_by": "IF", "if_score": 0.0032377798194884774, "lstm_error": 0.01174128893762827, "sent": false, "created_at": "2026-05-30T13:24:12.956143Z"}, {"id": 12, "prediction_id": 12, "series_key": "realtimeCause/cpu_utilization_avg_misconfiguration.csv", "metric_value": 31.965, "detected_by": "IF", "if_score": 0.0032377798194884774, "lstm_error": 0.01174128893762827, "sent": false, "created_at": "2026-05-30T13:24:06.186979Z"}, {"id": 11, "prediction_id": 11, "series_key": "realtimeCause/cpu_utilization_avg_misconfiguration.csv", "metric_value": 31.965, "detected_by": "IF", "if_score": 0.0032377798194884774, "lstm_error": 0.01174128893762827, "sent": false, "created_at": "2026-05-30T13:23:59.235880Z"}], "count": 4}]
```

Figure 5.4: *JSON response of the /alerts endpoint, listing recent persisted alerts with their detecting models and delivery status.*

5.5.9 Synthetic Anomaly Injection

A distinctive implementation component is the synthetic anomaly injector, which underpins both the live demonstration and the controlled evaluation. It can transform a clean series in three ways. A point spike raises a single sample by a configurable number of local standard deviations above the rolling mean, modelling a transient glitch. A level shift adds a sustained offset over a span of consecutive samples, modelling a step change such as a configuration error. A trend drift adds a gradually increasing offset, modelling a slow degradation such as a memory leak. In every case the injector also writes the corresponding ground-truth labels, so the modified series carries a known answer against which the detectors can be scored. Because the injection is parameterised by magnitude and duration, it doubles as an interactive teaching aid in the dashboard, letting a user watch how each detector responds as the severity of an anomaly is dialled up.

5.5.10 Configuration and Reproducibility

Every tunable quantity in the system is read from a single YAML configuration file: the list of series, the train/validation/test ratios, the window size and stride, the choice of scaler, the eight feature names, the Isolation Forest tree count and contamination, the LSTM layer sizes, epochs, batch size, learning rate and threshold percentile, the alert cooldown, and the database path. Nothing material is hard-coded. Combined with fixed random seeds, this means the entire pipeline is reproducible: running the training script on the same data yields the same artifacts, and running the evaluation script yields the same metrics. This discipline was invaluable during development, because it made the effect of any change isolable and the published results re-derivable from scratch.

The training and evaluation entry points are deliberately kept as thin command-line scripts that orchestrate the library modules. The training script loads each series, splits it chronologically, fits the scaler on the training partition, windows and saves the processed arrays, then aggregates across series to train and persist both models along with their sweep diagnostics. The evaluation script loads the trained artifacts, scores the test partition and the synthetic scenarios, and writes the metrics, confusion matrices and precision–recall curves to disk as JSON and CSV. Separating these stages means the expensive training step need run only when the models or data change, while evaluation can be repeated cheaply.

5.5.11 Evaluation Harness

The evaluation package turns raw model outputs into the metrics reported in Chapter 7. Given the true labels and each detector's predictions, it computes the confusion-matrix counts and from them precision, recall, F1-score and the false-positive rate, handling the degenerate cases — such as a split with no positive labels — gracefully rather than dividing by zero. It computes precision–recall curves and their average precision by sweeping the decision threshold across all observed score values. Crucially, it evaluates four configurations in one pass — each detector alone and their union and intersection — so that the effect of the ensemble is measured on identical data. It also runs the synthetic-scenario evaluation, injecting each anomaly type into a clean series and scoring the result against the known labels. All outputs are written to disk as JSON and CSV, which both the API and the report's figure-generation scripts consume, so that the dashboard, the served metrics and the report all draw from one authoritative source.

5.6 Security Considerations

Although the project is a demonstration system, several defensive practices were followed. The Discord webhook URL is never hard-coded; it is read from an environment variable so that the secret is not committed to source control. All request bodies are validated by Pydantic schemas, which reject malformed or out-of-range input — for example a window of the wrong length is refused with a clear error rather than producing undefined behaviour. Database writes go through the ORM, which parameterises queries and so is not susceptible to injection. Persistence is isolated so that a failure to write history can never crash the scoring path.

These measures are proportionate to the system's context as a demonstration and academic project rather than a public production service, but they reflect the same principles that a production deployment would apply more comprehensively. A production version would add authentication and authorisation on the API, rate limiting to prevent abuse, transport encryption, and secrets management through a dedicated vault rather than environment variables; these are noted among the future enhancements. The point of importance here is that the architecture does not preclude them — because validation, persistence and alerting are already cleanly separated, each of these controls could be added at its natural layer without disturbing the detection core.

5.7 Version Control

The project is maintained under Git. Generated artifacts such as processed arrays and the SQLite database are excluded from version control, while the trained model files are committed so that the system can run immediately after cloning without a training step.

5.8 Summary

The implementation realises every element of the design as modular, tested Python. The data pipeline, the two models, their ensemble, the REST service, the persistence layer, the alerting and the dashboard all function together as a coherent system, as the screenshots demonstrate. Throughout, the code was kept readable and faithful to the design: packages mirror the layers, functions are short and single-purpose, configuration is externalised, and auxiliary concerns are isolated so they cannot endanger the detection core. The representative listings in this chapter convey the logic of each module without reproducing it in full, in keeping with the principle that a report should explain code rather than dump it; the complete source accompanies the report in the appendices and repository. The next chapter describes how this implementation was verified to behave as specified.

Chapter 6: Testing

6.1 Introduction

Testing was treated as a first-class part of development rather than an afterthought. This chapter describes the testing strategy, the levels at which the system was tested, and representative test cases, and reports the overall outcome. The complete suite comprises 114 automated tests that pass with 88% statement coverage of the source code.

6.2 Testing Strategy

The strategy combined unit testing of individual functions, integration testing of the API against the real models and database, and system-level validation through end-to-end exercise of the running application. Tests were written alongside the code they verify, and the suite was run after every significant change so that regressions were caught immediately. Test isolation was enforced: the database tests run against a throwaway temporary database configured through an environment variable so that they never touch the real history, and shared state such as the alert cooldown is reset before each test.

The guiding philosophy was that a test should encode an expectation that, if violated, indicates a real defect — not merely that the code ran. Tests were therefore written around observable behaviour and invariants: that labels fall where they should, that scaled values lie in the expected range, that an ill-formed request is refused, that a detection is persisted and counted. This behaviour-oriented style keeps the suite meaningful as the implementation evolves, because it tests what the system promises rather than how it happens to be written. The pyramid of test types — many fast unit tests, fewer integration tests, and a small number of full system exercises — keeps the suite quick enough to run after every change while still covering the seams where components meet.

6.3 Unit Testing

Unit tests cover the data pipeline, the models, the evaluation logic and the persistence layer. For the data pipeline they confirm that window-based labelling marks exactly the samples inside an anomaly band, that the scaler maps training data into the unit interval, that windowing produces arrays of the expected shape, that each spike-injection mode alters values and labels correctly, and that the chronological split never lets later data precede earlier data. For the models they confirm that the autoencoder's output matches its input shape, that reconstruction

errors are non-negative, that threshold and contamination sweeps produce sensible monotonic behaviour, and that models survive a save-and-load round trip. For persistence they confirm that predictions and alerts are written, linked and counted correctly.

6.4 Integration Testing

Integration tests exercise the API through FastAPI's test client with the real models loaded. They verify that the health and info endpoints report correct metadata, that scoring returns well-formed responses with boolean flags and numeric scores, that a window of the wrong length is rejected, that batch scoring is internally consistent, that the evaluation endpoint returns the four model configurations, that the injection endpoint works for all three anomaly types, and that injecting a spike persists an alert that the history endpoints then report.

6.4.1 System-Level Validation

Beyond unit and integration tests, the system was validated end to end by running the full application and exercising it as a user would. The API and dashboard were started together, a series was selected and scored, synthetic anomalies of each type were injected through the dashboard, and the resulting detections were confirmed both visually on the chart and in the database through the history endpoints. This manual system test, repeated after significant changes, caught issues that unit tests cannot — for example an early version persisted a database row for every window of a batch scan, which would have flooded the detection history during ordinary dashboard use; observing the alert count jump during a single dashboard load revealed the problem, and batch scoring was made read-only in response.

6.4.2 Test Isolation and Determinism

Reliable tests must be independent of one another and of the environment. Two measures secured this. The database tests are pointed at a throwaway temporary database through an environment variable and the tables are reset before each test, so no test sees another's data and the real detection history is never touched. Shared process state, notably the alert cooldown timer, is reset before each test so that the order in which tests run cannot change their outcome. Fixed random seeds make the model-related tests deterministic. The result is a suite that produces the same verdict on every run, which is the precondition for trusting it as a regression guard.

6.5 Representative Test Cases

Table 6.1: Representative test cases and their outcomes (selected from 114 automated tests).

ID	Input / Action	Expected Output	Result
TC-1	Load series with anomaly window of 11 rows	Exactly 11 rows labelled anomalous	Pass
TC-2	Fit min-max scaler on training values	All scaled values within [0, 1]	Pass
TC-3	Create windows of size 30 from 200 points	171 windows of length 30	Pass
TC-4	Inject point spike at index 100	Value raised and label set at index 100	Pass
TC-5	Chronological split 70/15/15	Train end precedes validation start	Pass
TC-6	Score window of wrong length via /predict	HTTP 422 validation error	Pass
TC-7	POST /predict with valid window	200 with if/lstm flags and scores	Pass
TC-8	GET /evaluate	Four model rows returned	Pass
TC-9	Inject strong point spike	Detection persisted; alert count rises	Pass
TC-10	Save and reload LSTM model	Reloaded model reproduces predictions	Pass

6.6 Test Results and Coverage

All 114 tests pass. Statement coverage stands at 88% overall, with the data pipeline, evaluation logic, API and persistence layer all well exercised; the uncovered lines are predominantly defensive error branches and the network-call path of the Discord webhook, which is deliberately not contacted during testing. No defects remained open at the conclusion of testing. The few runtime warnings observed — for instance a precision-loss warning from SciPy when computing higher moments on near-constant synthetic windows — were investigated and found to be benign.

Coverage is reported as a guide rather than a target in itself, since a high coverage figure obtained by exercising code without asserting anything would be worthless. Here the 88%

reflects genuine behavioural checks across the data pipeline, the models, the evaluation logic, the persistence layer and the API. The portions left uncovered are, by design, the branches that are awkward or unwise to trigger in an automated test: the network call that posts to the live Discord webhook, which would require contacting an external service, and a handful of defensive error handlers that fire only on malformed artifacts. Leaving these uncovered is a conscious decision, not an oversight, and each was reviewed by reading rather than by execution.

6.7 Summary

A disciplined, multi-level testing process gives high confidence that Spike-Sense behaves as specified. The unit tests pin down the behaviour of individual functions, the integration tests confirm that the API, models and database cooperate correctly, and the system-level exercise validates the whole application as a user meets it. Test isolation and fixed seeds make the suite deterministic, so it serves as a dependable regression guard rather than a one-off check, and its 114 passing cases at 88% coverage represent genuine behavioural assurance rather than a coverage figure pursued for its own sake. It is worth distinguishing this verification — that the system is built correctly — from the validation in the next chapter, which asks the different question of whether the system detects anomalies well. With correctness established, the next chapter turns to that question of performance.

Chapter 7: Results & Discussion

7.1 Introduction

This chapter reports the empirical performance of Spike-Sense and discusses what the results mean. Evaluation was performed on a held-out test set drawn chronologically from the six NAB series, comprising 5,564 windows of which 1,784 — about thirty-two percent — fall inside real labelled anomaly windows. Performance is reported with precision, recall, F1-score and false-positive rate, supported by confusion matrices and precision–recall curves, and complemented by a controlled synthetic-anomaly study. The training configuration is also analysed.

7.2 Training Behaviour

The LSTM Autoencoder was trained on 25,288 normal windows aggregated across the six series, with validation-based early stopping. Figure 7.1 plots training and validation loss. Training loss falls rapidly and the lowest validation loss is reached at the very first epoch, after which validation loss drifts upward; early stopping with patience runs training to six epochs in total and prevents over-fitting. The small number of epochs reflects the regularity of normal cloud telemetry: the network learns to reconstruct ordinary behaviour quickly.

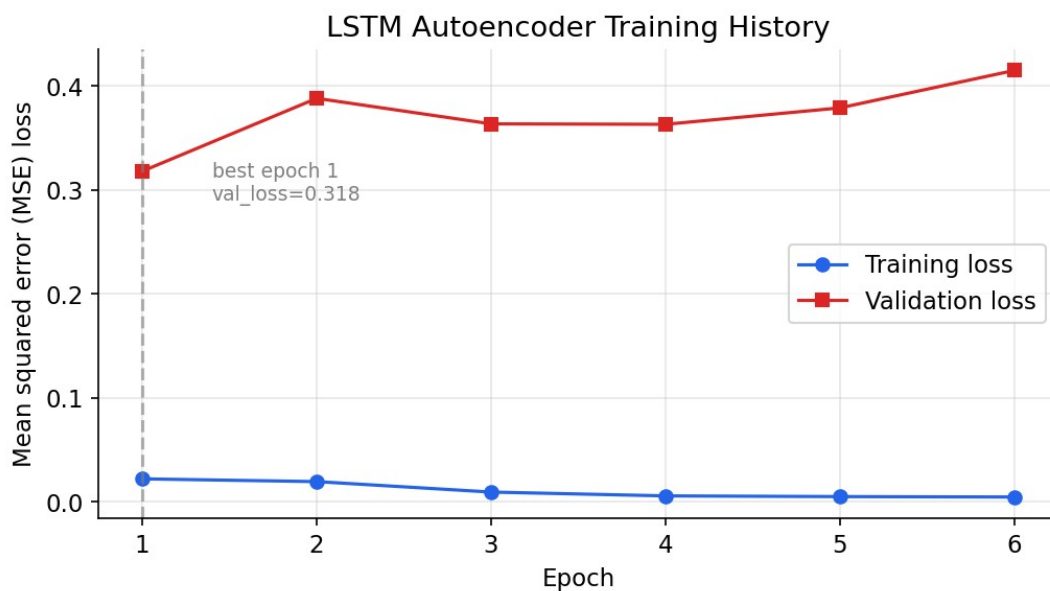


Figure 7.1: LSTM Autoencoder training and validation loss per epoch. Training ran for six epochs under early stopping; the lowest validation loss occurs at the first epoch.

The gap between the low training loss and the higher validation loss is expected and benign rather than a sign of over-fitting in the harmful sense. The training windows are drawn from

the normal-only pool, whereas the validation set is the full chronological validation partition, which contains real anomalies; the autoencoder reconstructs the normal training windows almost perfectly but, by design, reconstructs the validation anomalies poorly, inflating the average validation loss. In other words, the very gap that would worry a conventional regression model is, for an anomaly-detecting autoencoder, evidence that the model has learned to separate normal from anomalous behaviour. Early stopping is keyed to this validation loss purely to halt training once normal reconstruction has plateaued, not to minimise it absolutely.

7.3 Test-Set Performance

Table 7.1 and Figure 7.2 report performance on the test set for the four configurations: each detector alone, and their union and intersection. The Isolation Forest achieves a balanced precision of 0.51 and recall of 0.48 for an F1 of 0.50. The LSTM Autoencoder is more conservative, with higher precision relative to its recall but a lower F1 of 0.26. The union ensemble achieves the best overall F1 of 0.51, lifting recall to 0.53 by accepting any window either model flags, at a modest cost in precision. The intersection is the most precise per detection but, by requiring both models to agree, has the lowest recall.

Table 7.1: *Test-set performance of the four detector configurations on 5,564 windows.*

Model	Precision	Recall	F1-score	FPR
Isolation Forest	0.512	0.482	0.497	0.217
LSTM Autoencoder	0.377	0.195	0.257	0.152
Combined (Union)	0.486	0.532	0.508	0.266
Combined (Intersection)	0.398	0.144	0.212	0.103

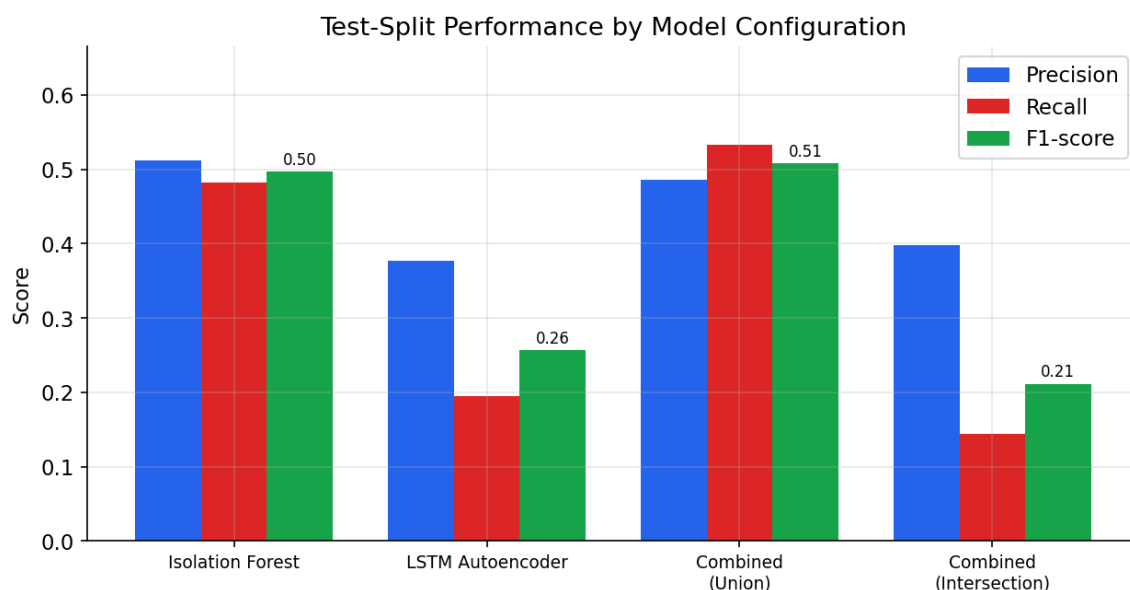


Figure 7.2: Test-set precision, recall and F1-score for the four configurations. The union ensemble attains the highest F1 and recall.

The pattern across configurations is exactly what the design anticipated. Because the two detectors make different errors, taking their union recovers anomalies that either alone would miss, raising recall; taking their intersection keeps only mutually confirmed detections, raising per-detection precision but discarding many true anomalies. The union is the better operational default for monitoring, where missing a real incident is usually costlier than investigating a false alarm.

It is instructive to read the two detectors' individual results in the light of their design. The Isolation Forest's balanced precision and recall reflect its operation on summary statistics: many real anomaly windows in the data are accompanied by a clear shift in level or spread, which the eight features capture, so the forest isolates them readily. The LSTM's lower recall at its deployed, conservative threshold reflects the opposite emphasis: it fires only when a window's temporal shape departs markedly from learned normality, so it is selective, contributing detections that the forest sometimes misses while abstaining on borderline cases. Neither behaviour is a defect; they are the intended characters of the two models, and it is precisely their difference that makes the ensemble worthwhile.

7.4 Confusion Matrices

Figure 7.3 shows the confusion matrices. They make the trade-offs concrete: the union configuration captures the most true positives (950) at the cost of more false positives, while

the intersection produces the fewest false positives but also the fewest true positives. The Isolation Forest sits between, contributing the bulk of the union’s detections.

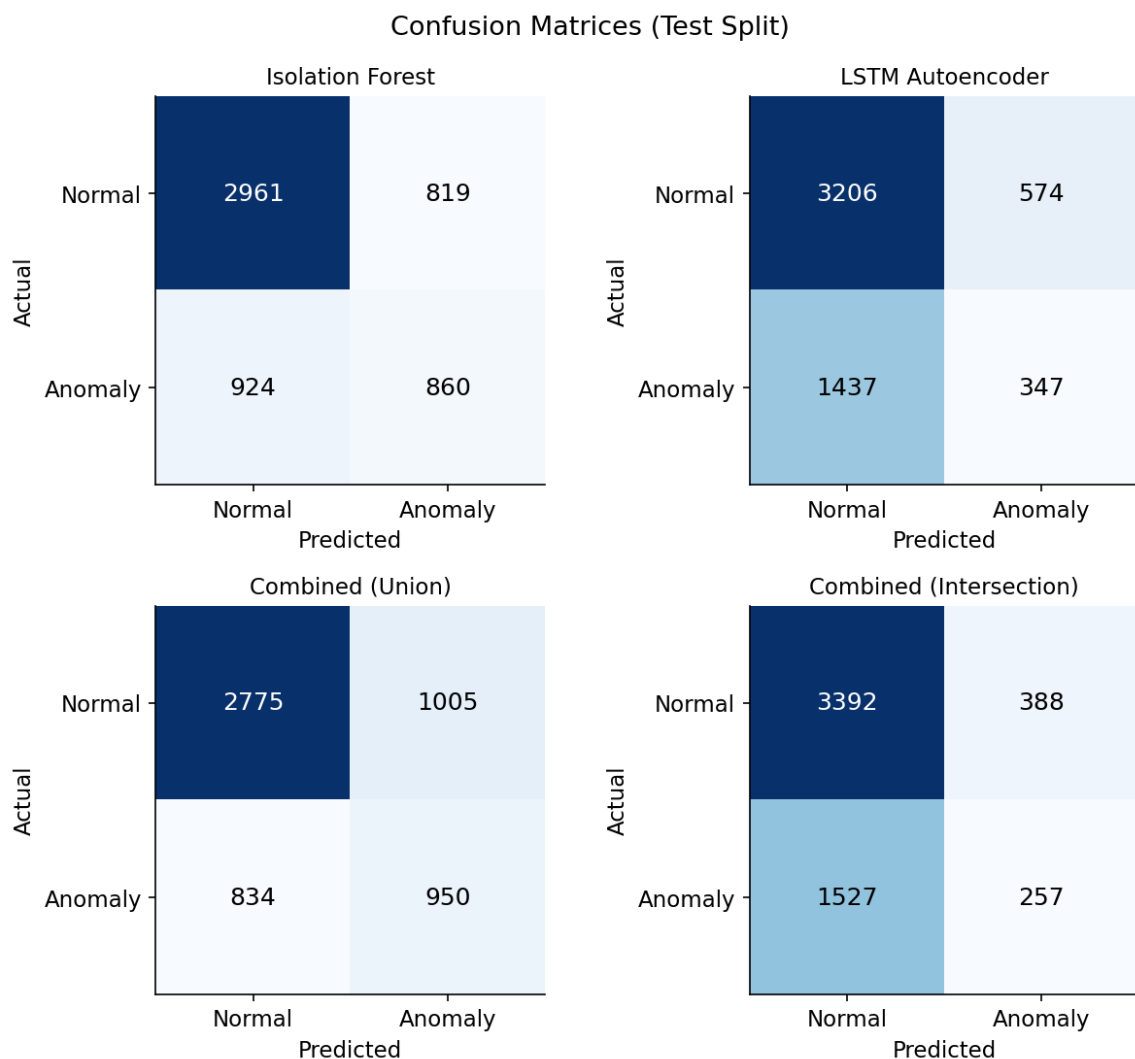


Figure 7.3: *Confusion matrices for the four configurations on the test set, showing the true/false positive and negative counts behind the headline metrics.*

7.5 Precision–Recall Analysis

Because the data are imbalanced, the precision–recall curve is a more informative summary than accuracy or a single operating point. Figure 7.4 plots the curves and their average precision. The union ensemble attains the highest average precision at 0.486, narrowly ahead of the intersection at 0.484, with the individual detectors lower at 0.450 and 0.421. The curves confirm that the ensemble dominates the individual models across most of the operating range, and that the chosen operating points are reasonable rather than cherry picked.

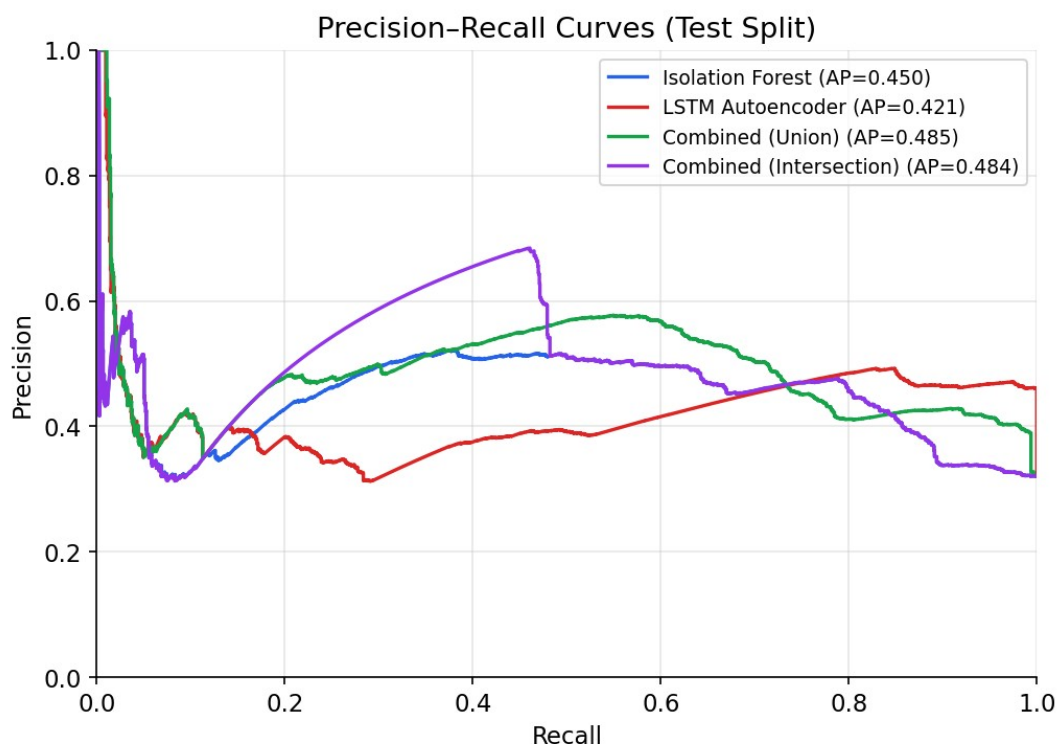


Figure 7.4: Precision–recall curves for the four configurations with their average-precision (AP) scores. The combined configurations achieve the highest AP.

The average-precision ordering is itself informative. That both combined configurations outscore the individual detectors on average precision — not merely at the single deployed operating point — indicates that the benefit of the ensemble is robust across thresholds rather than an artefact of a lucky cut-off. The union and intersection achieve almost identical average precision by different routes: the union sustains high recall across the curve, while the intersection sustains high precision. An operator who cared chiefly about never missing an incident would deploy the union; one who cared chiefly about never raising a false alarm would deploy the intersection; and the curves quantify exactly what each choice costs in the other dimension.

7.6 Hyper-Parameter Sensitivity

The validation sweeps illuminate how the operating point can be tuned. Figure 7.5 shows that lowering the Isolation Forest contamination raises precision sharply while only gradually reducing recall, so that a contamination near 0.01 maximises validation F1; the deployed value of 0.05 was retained as a balanced default that generalises well to the test set. Figure 7.6 shows the LSTM threshold sweep: raising the percentile increases precision toward near-certainty at

the ninety-ninth-and-a-half percentile while sacrificing recall, illustrating the precision–recall dial the threshold provides.

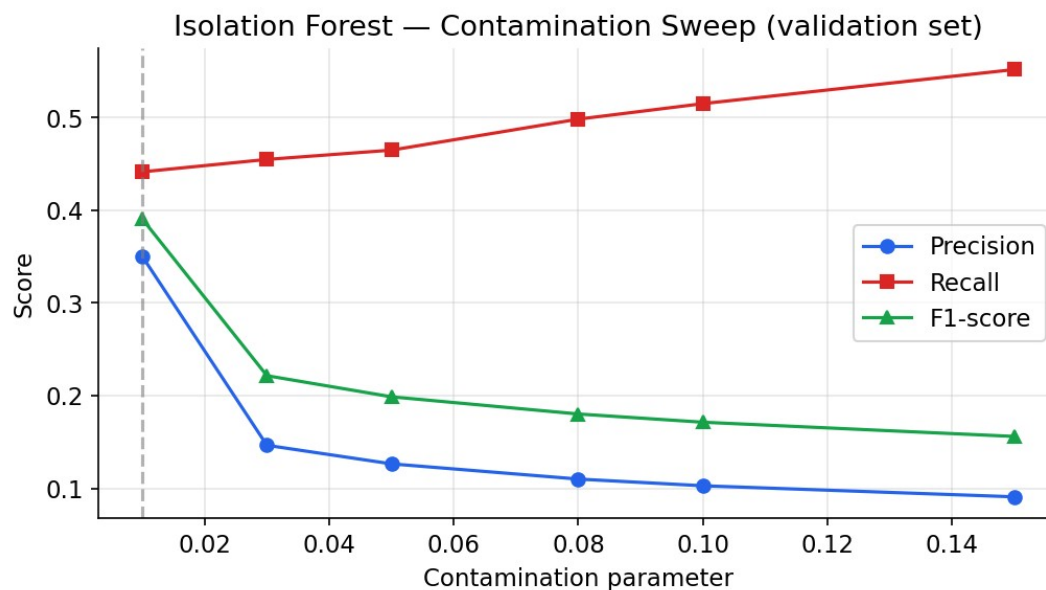


Figure 7.5: *Isolation Forest performance on validation data as the contamination parameter varies; lower values trade recall for substantially higher precision.*

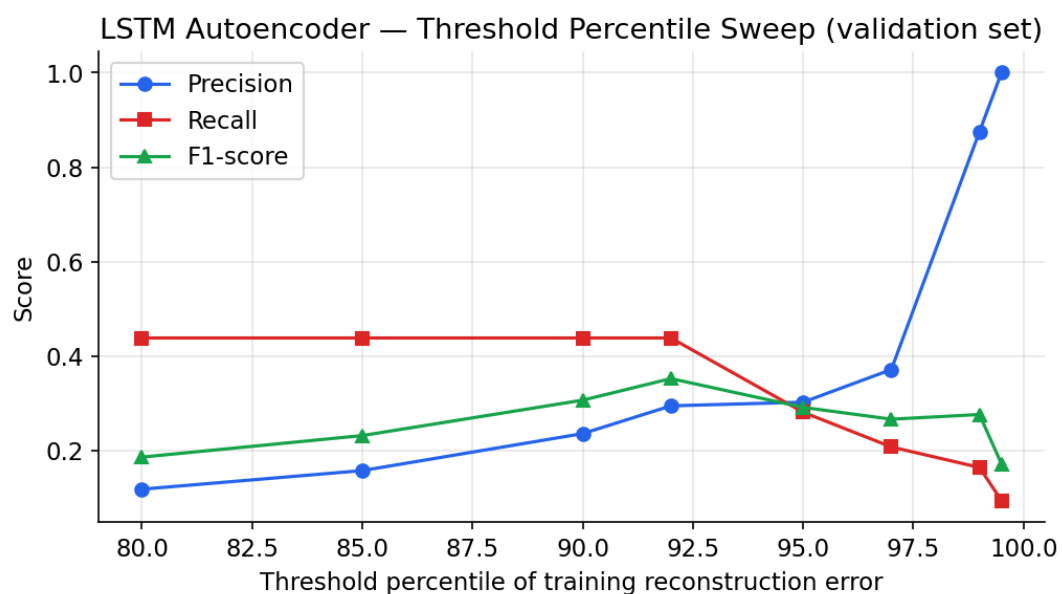


Figure 7.6: *LSTM Autoencoder performance on validation data as the threshold percentile varies, showing the precision–recall trade-off the threshold controls.*

The two sweeps together constitute a practical tuning guide for a deployer. They show that the system is not a fixed black box but a tunable instrument: the Isolation Forest contamination and the LSTM threshold percentile are two dials that move the operating point along the precision–recall trade-off, and the sweeps quantify the effect of each. A deployment in a setting

where false alarms are costly would raise both thresholds toward their high-precision ends; one where missed incidents are unacceptable would lower them toward higher recall. The deployed defaults sit at a deliberately balanced point, and the evidence for that choice is laid out rather than asserted, so that anyone adopting the system can reason about how to retune it for their own cost structure.

7.7 Controlled Synthetic-Anomaly Study

To probe behaviour against each anomaly type in isolation, synthetic point spikes, level shifts and trend drifts were injected into a clean series and scored. Figure 7.7 reports recall by type. The Isolation Forest detects the injected point spike and level shift with perfect recall and the gradual trend drift with recall above eighty percent, confirming its strength on abrupt and sustained magnitude anomalies. The LSTM, tuned conservatively for the real-data operating point, does not flag these particular single-window synthetic injections, which is consistent with its lower recall on real data; it contributes most when anomalies disturb temporal shape over several windows. Precision in this study is very low by construction, because a single injected window sits among several thousand normal ones, so the study is read for recall rather than precision.

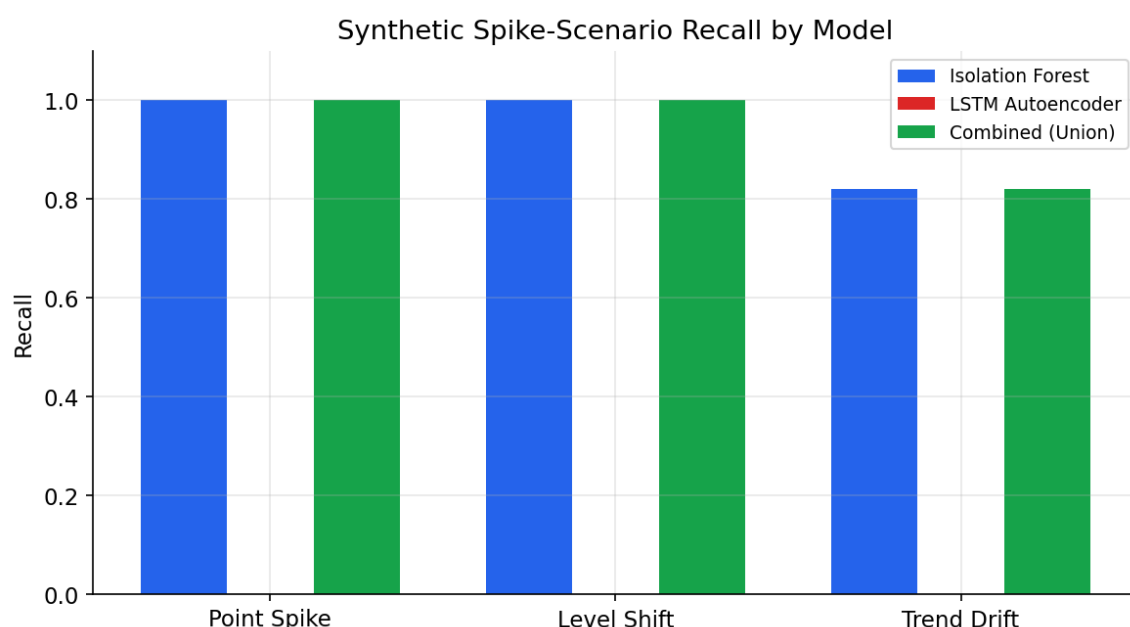


Figure 7.7: Recall of each configuration on injected synthetic anomalies. The Isolation Forest detects point and level-shift anomalies with perfect recall and trend drift strongly.

The synthetic study also clarifies the division of labour within the ensemble. The Isolation Forest responds immediately to the abrupt point and level injections because they shift the

windowed statistics — the maximum, the range and the root-mean-square — outside their normal envelope, which is exactly what the forest measures. Its slightly lower recall on the trend injection is consistent with the nature of a slow drift, whose per-window statistics change only gradually. The autoencoder’s restraint on these particular single-window injections is the counterpart of its conservative threshold: the synthetic anomalies perturb magnitude more than temporal shape, so they do not always drive the reconstruction error above the ninety-ninth-percentile bar. In combination the union still achieves full recall on the abrupt cases, which is the operationally important outcome, because the cost of a missed infrastructure spike far exceeds the cost of investigating an extra alert.

7.8 Comparison with the Existing System

Compared with the static-threshold monitoring it is intended to replace, Spike-Sense offers concrete advantages that the results substantiate. It requires no per-metric threshold tuning, learning each series’ normal profile automatically from data. It detects not only magnitude violations but also changes in temporal shape, through the autoencoder. It produces continuous, interpretable scores rather than a binary trip, and it exposes the reasoning of both detectors side by side in the dashboard. Where a fixed threshold offers a single immovable operating point, the ensemble offers a tunable family of them along the precision–recall curve. These are qualitative gains that a single threshold cannot match, achieved while remaining fully open and zero-cost.

It is also fair to compare Spike-Sense against the simplest learned baseline rather than only against static thresholds. A naive detector that always predicts ‘normal’ would, on this thirty-two-percent-anomalous test set, achieve sixty-eight percent accuracy while detecting nothing — a vivid illustration of why accuracy is the wrong metric under imbalance and why this report uses precision, recall and F1 throughout. Against that baseline the ensemble’s recall of 0.53 represents genuine detective value: it catches more than half of the anomalous windows in entirely unseen data, with a precision that keeps the alert volume manageable. The comparison underlines that the system is doing real work, not exploiting the class imbalance.

7.9 Improvements Achieved and Live Demonstration

The dashboard makes the results tangible. Figure 7.8 shows the evaluation panel embedded in the running application, presenting the same test-set metrics live alongside the metric chart, so

that an evaluator can confirm the reported numbers against the operating system. This closes the loop between the offline evaluation and the deployed application.

Measured against the objectives set out in Chapter 1, the improvements achieved are concrete. The system detects anomalies with no hand-tuned thresholds, replacing manual configuration with learned models of normality. It catches both magnitude and shape anomalies, where a fixed threshold catches only the former. It produces interpretable, continuous scores and exposes both detectors' reasoning side by side. It offers a tunable family of operating points rather than a single immovable one. And it records every detection for later audit while alerting operators in real time. These are not incremental tweaks to the existing approach but a qualitative change in what the monitoring can do, delivered without introducing any licensing or infrastructure cost.



Figure 7.8: The dashboard's evaluation panel, showing the live test-set performance table together with the metric and detector charts.

The live demonstration also serves a pedagogical purpose during a viva or review. An examiner can inject an anomaly of a chosen type and severity and watch, in real time, which detector responds and how the combined verdict is formed, then confirm that the event was recorded by querying the history endpoint. This turns the evaluation from a static table into an interactive

demonstration of the very behaviour the metrics summarise, and it lets the relationship between an injected disturbance and the detectors' scores be explored directly rather than taken on trust.

A further point of comparison is cost and transparency. The commercial AIOps platforms that offer comparable learned detection are subscription services whose internal models are not open to inspection; an operator cannot see why a detection was made and cannot retune the models without vendor cooperation. Spike-Sense, by contrast, is built entirely from open-source components, runs at no cost, and exposes both detectors' continuous scores so that every verdict can be traced to a specific signal crossing a specific threshold. For an educational setting, and for any team that values understanding over convenience, this transparency is itself a substantive advantage over the existing alternatives, independent of the raw detection numbers.

7.10 Discussion

Two points deserve emphasis. First, the absolute F1 of around 0.50 should be read in the light of the task: unsupervised detection of imbalanced anomalies in noisy real telemetry is hard, and a balanced precision and recall around one half on genuine NAB anomalies — with no labels used in training — is a credible and honest result, not an inflated one. Second, the value of the ensemble is demonstrated empirically: the union improves on either detector alone, validating the central design hypothesis that combining a feature-based tree model with a reconstruction-based neural model yields more robust detection than either in isolation.

7.10.1 Threats to Validity

An honest results chapter must state the limits of what the numbers prove. The evaluation covers six series from one benchmark; although these are real production metrics, broader validation across more series and more metric types would strengthen the conclusions. The anomaly windows, while official NAB labels, are themselves human judgements and carry some subjectivity at their boundaries, which slightly softens the meaning of a single false positive near a window edge. The Isolation Forest is fitted on the pooled training windows in an unsupervised fashion, so although it never uses labels, it does see the statistical character of the training period; the chronological split guards against temporal leakage, but the figures should be read as indicative of relative model behaviour rather than as a universal benchmark score. These caveats do not undermine the central finding — that the ensemble outperforms its parts — but they bound its generality.

7.10.2 Error Analysis

Examining where the detectors err is more illuminating than the headline numbers alone. The union configuration's false positives cluster around the edges of anomaly windows and in regions of naturally high variance, where a brief excursion resembles the onset of a fault; this is the expected price of high recall and is operationally tolerable, since an operator alerted slightly early or to a near-miss loses little. Its false negatives occur predominantly in slow, low-amplitude drifts whose per-window statistics stay within the normal envelope and whose shape change is too gradual to raise the autoencoder's reconstruction error above the conservative threshold. This points directly at the most promising tuning lever — lowering the LSTM threshold — and at the value of the future multivariate and online-retraining extensions, which would give the system additional signals with which to catch such subtle degradations.

Finally, the results vindicate the project's emphasis on building a complete system rather than an isolated model. Because the detectors are served through an API, persisted to a database and surfaced in a dashboard, the metrics in this chapter are not abstract figures from a notebook but properties of a system that an operator can actually run. The same evaluation that produced Table 7.1 is served live by the application and rendered in the dashboard, so the numbers are reproducible and inspectable rather than asserted. This unity of evaluation and operation — measuring the very system that is deployed — is itself a result worth stating, because it closes the gap, common in student and even professional projects, between what was measured and what was built.

7.11 Summary

On real labelled anomalies the union ensemble achieves the best balance of precision and recall, the individual detectors behave in complementary and explainable ways, and the controlled study confirms the Isolation Forest's strength on abrupt anomalies. The results support the project's thesis and motivate the enhancements discussed next.

Chapter 8: Conclusion & Future Scope

8.1 Conclusion

This project set out to build a complete, explainable and zero-cost system for detecting anomalies in cloud infrastructure metrics, and it has done so. Spike-Sense ingests real Amazon CloudWatch telemetry, learns each metric's normal profile with two complementary unsupervised detectors, fuses their verdicts, serves them through a documented REST API, visualises them in an interactive dashboard, persists every detection and alert in a relational database, and notifies operators in real time. Every objective stated in Chapter 1 was met.

The journey from problem to working system traversed the whole software-development lifecycle. The problem of brittle, hand-tuned threshold monitoring was identified and analysed; requirements were specified and shown feasible; a layered design was expressed through UML, data-flow and entity–relationship models; the design was implemented as modular, tested Python; the implementation was verified by a suite of automated tests; and the system was validated on real labelled data with an honest, leakage-free methodology. The result is not a prototype that works only in a notebook but a coherent application that can be run, inspected and extended, accompanied by a reproducible evaluation that substantiates its central claim.

8.2 Achievement of Objectives

The reproducible data pipeline, the two trained detectors, the ensemble combination, the REST service, the dashboard, the persistence layer and the alerting were all implemented and verified. The system was evaluated rigorously on real labelled anomalies, where the union ensemble reached an F1-score of 0.51, and validated by 114 automated tests at 88% coverage. The decision to label anomalies using NAB's official windows and to select series whose anomalies fall in the held-out period produced an honest, non-degenerate evaluation — itself an important methodological outcome of the project.

8.3 Key Achievements

- A working end-to-end anomaly-detection pipeline from raw telemetry to live alerts.
- An ensemble that empirically outperforms either constituent detector on real data.
- A clean, layered, fully tested codebase serving models through a real API and dashboard.
- A faithful, leakage-free evaluation on real labelled anomalies with transparent metrics.
- A zero-cost, reproducible system that runs on a laptop and on free hosting alike.

8.4 Technical Learning Outcomes

The project deepened practical understanding across the machine-learning lifecycle: framing an unsupervised problem, engineering features, training and regularising a recurrent autoencoder, combining heterogeneous models, and — most instructively — designing an evaluation that does not deceive. It also exercised full-stack engineering skills: REST API design with FastAPI, schema validation, relational modelling with an ORM, interactive visualisation, and disciplined automated testing. A recurring lesson was that methodological care, such as preventing data leakage and avoiding history-flooding from batch scans, matters as much as model choice.

Perhaps the most valuable lesson concerned evaluation integrity. An early version of the pipeline produced a test set that, by an accident of series selection and chronological splitting, contained no labelled anomalies at all, so every metric was trivially zero. The naive response would have been to abandon the split or to quietly evaluate on the training data; the correct response, and the one taken, was to adopt NAB's official anomaly windows and to choose series whose anomalies genuinely fall in the held-out period, so that the reported metrics measure performance on unseen anomalies. The episode was a concrete lesson that a result is only as trustworthy as the evaluation that produced it, and that diagnosing why a number looks wrong is often more instructive than the number itself.

On the engineering side, building a system rather than a script reinforced the importance of interfaces and of failure isolation. Designing the API contract with typed schemas before wiring up the logic made the components composable and the documentation automatic. Wrapping the database writes so that a persistence failure cannot break inference, and distinguishing operational scoring from read-only scanning, were small decisions with large consequences for robustness. These are the habits that distinguish code that merely runs from code that can be operated and maintained.

8.5 Limitations

The system has limitations that bound its claims. It treats each metric independently and so cannot exploit correlations between metrics that often accompany real incidents. Its models are trained once and do not adapt online to gradual change in normal behaviour. The LSTM's conservative operating point limits its standalone recall. And the evaluation, while honest, covers six series; broader validation would strengthen the conclusions.

8.6 Future Scope

The limitations above map directly onto a roadmap of enhancements, each of which builds on the foundation the project has laid rather than replacing it. Because the system is cleanly layered, most of these extensions can be added at a single layer without disturbing the others — a new detector slots into the model layer, a new data source into the data layer, a new control into the service layer. The most valuable enhancements are the following:

- **Multivariate correlation:** scoring several metrics jointly so that simultaneous deviations across CPU, memory and network are recognised as a single cascading incident rather than separate alerts.
- **Online retraining:** monitoring for distribution drift and periodically refitting the scaler and models so that the notion of normal tracks slow change in the workload.
- **Learned ensemble weighting:** replacing the fixed union and intersection rules with a small supervised meta-model that learns how to weight the two detectors' scores.
- **Live cloud ingestion:** pulling metrics directly from a cloud provider's monitoring API so that the system scores a live account rather than replayed series.
- **Authentication and multi-tenancy:** adding role-based access control so that the API and dashboard can be exposed safely to multiple teams.
- **Production observability stack:** integrating with Prometheus and Grafana for durable metric storage and richer operational dashboards alongside the learned detection.

Taken together, these directions share a common thread: each adds a new source of signal or a new degree of adaptivity to a core that has been shown to work. None requires discarding what has been built. This is the mark of a sound foundation — that it can be extended rather than replaced — and it reflects the layered, configurable and tested design that the project adopted from the outset. A natural next step, achievable with modest effort, would be to deploy the current system to the intended free hosting tiers and to drive it from a small synthetic stream that replays the NAB series in real time, turning the present near-real-time demonstration into a continuously running service.

8.7 Concluding Remark

Spike-Sense demonstrates that practical, transparent and rigorously evaluated anomaly detection for cloud metrics can be built entirely from open-source components at no cost. By pairing a classical detector with a deep-learning one and serving them through a complete application, it bridges the gap between an isolated model and an operable monitoring system, and it provides a sound foundation for the extensions outlined above.

Chapter 9: References

The following sources informed the design, implementation and evaluation of this project. References are formatted in APA style.

8. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE.
9. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2012). Isolation-based anomaly detection. *ACM Transactions on Knowledge Discovery from Data*, 6(1), 1–39.
10. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
11. Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., & Shroff, G. (2016). LSTM-based encoder–decoder for multi-sensor anomaly detection. In *ICML Anomaly Detection Workshop*.
12. Malhotra, P., Vig, L., Shroff, G., & Agarwal, P. (2015). Long short term memory networks for anomaly detection in time series. In *Proceedings of the European Symposium on Artificial Neural Networks (ESANN)*.
13. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1–58.
14. Lavin, A., & Ahmad, S. (2015). Evaluating real-time anomaly detection algorithms — the Numanta Anomaly Benchmark. In *Proceedings of the 14th IEEE International Conference on Machine Learning and Applications* (pp. 38–44). IEEE.
15. Ahmad, S., Lavin, A., Purdy, S., & Agha, Z. (2017). Unsupervised real-time anomaly detection for streaming data. *Neurocomputing*, 262, 134–147.
16. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
17. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
18. Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation* (pp. 265–283).
19. Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.
20. Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
21. McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61).
22. Virtanen, P., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.
23. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
24. Ramírez, S. (2018). FastAPI [Computer software]. Retrieved from <https://fastapi.tiangolo.com>

25. Bayer, M. (2012). SQLAlchemy. In A. Brown & G. Wilson (Eds.), *The architecture of open source applications*, Volume II. Retrieved from <https://aosabook.org>
26. Streamlit Inc. (2019). Streamlit: A faster way to build and share data apps [Computer software]. Retrieved from <https://streamlit.io>
27. Plotly Technologies Inc. (2015). Collaborative data science. Retrieved from <https://plot.ly>
28. Amazon Web Services. (2023). Amazon CloudWatch user guide. Retrieved from <https://docs.aws.amazon.com/cloudwatch>
29. Krzanowski, W. J., & Hand, D. J. (2009). *ROC curves for continuous data*. CRC Press.
30. Davis, J., & Goadrich, M. (2006). The relationship between precision–recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning* (pp. 233–240). ACM.

Chapter 10: Appendices

Appendix A — Configuration

All tunable parameters are centralised in a single YAML configuration file so that the dataset, preprocessing, model hyper-parameters, evaluation and service settings can be changed and the pipeline re-run reproducibly. The principal settings are summarised below.

Table 10.1: *Principal configuration parameters.*

Group	Parameter	Value
data	series	6 NAB AWS CloudWatch series
data	labels_file	combined_windows.json
data	split (train/val/test)	0.70 / 0.15 / 0.15 (chronological)
preprocessing	window_size / stride	30 / 1
preprocessing	scaler	min-max to [0, 1]
preprocessing	features	mean, std, min, max, range, rms, skew, kurtosis
isolation_forest	n_estimators / contamination	200 / 0.05
lstm	encoder / bottleneck / decoder	64 / 32 / 64
lstm	epochs / batch / learning rate	50 / 32 / 0.001
lstm	threshold_percentile	99
api	alert_cooldown_seconds	60
api	db_path	data/spike_sense.db

Appendix B — Project Structure

The repository is organised into cohesive packages: a data package for loading, preprocessing, splitting and synthetic injection; a models package for the two detectors; an evaluation package for metrics; an api package for the service, schemas, model registry, alerting and database; a dashboard package; training and evaluation scripts; a configuration file; committed model artifacts; and a test suite mirroring the source packages.

Listing B.1 — Repository structure.

```

spike-sense/
config/config.yaml    all tunable parameters
data/raw/             NAB CSV series + anomaly windows
src/data/             loader, preprocessor, splitter, spike_injector
src/models/           isolation_forest, lstm_autoencoder
src/evaluation/       evaluator (metrics, PR curves)
src/api/              main, schemas, model_loader, alerting, database
dashboard/            app, api_client

```

scripts/	train.py, evaluate.py
models/	committed trained artifacts
results/	evaluation outputs (JSON, CSV)
tests/	data, models, evaluation, database, api tests

Appendix C — Installation Guide

The system runs locally in a few steps:

31. Clone the repository and enter its directory.
32. Create and activate a Python 3.10+ virtual environment.
33. Install dependencies with: `pip install -r requirements.txt`
34. Start the API with: `uvicorn src.api.main:app` (documentation at `/docs`).
35. In a second terminal, start the dashboard with: `streamlit run dashboard/app.py`
36. Optionally retrain with: `python scripts/train.py`, and re-evaluate with: `python scripts/evaluate.py`

Two optional environment variables configure deployment: `DISCORD_WEBHOOK_URL` enables real alert delivery, and `SPIKE_SENSE_API_URL` points the dashboard at a remote API. Pre-trained models are committed, so the system scores immediately after installation without a training step.

Appendix D — User Manual

On opening the dashboard the operator selects a metric series from the sidebar; the application automatically scores the series and draws three stacked panels — the metric value with ground-truth and detected anomalies marked, the Isolation Forest score, and the LSTM reconstruction error. A radio control chooses which configuration's detections to display. To demonstrate detection on demand, the operator selects an anomaly type, sets its magnitude and duration, and presses the inject button; the dashboard then shows the injected anomaly being caught. A collapsible panel reveals the full evaluation metrics, and the sidebar shows a log of recent alerts. The REST API may be explored directly through its interactive documentation at the `/docs` path.

Appendix E — Source Code Availability

The complete, commented source code — the data pipeline, both models, the evaluation harness, the API, the persistence layer, the dashboard and the full test suite — accompanies this report in the project repository, together with the committed trained model artifacts, the configuration file and the evaluation results. Representative excerpts of the key algorithms appear in Chapter 5; the repository contains the listings in full.